

UNIVERZITA PARDUBICE  
FAKULTA ELEKTROTECHNIKY A  
INFORMATIKY

BAKALÁŘSKÁ PRÁCE

2026

Maksim Zabrodin

Univerzita Pardubice  
Fakulta elektrotechniky a informatiky

Aplikace pro zlepšení finanční gramotnosti a správu osobních financí  
Bakalářská práce

Univerzita Pardubice  
Fakulta elektrotechniky a informatiky  
Akademický rok: 2024/2025

# ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

|                     |                                                                             |
|---------------------|-----------------------------------------------------------------------------|
| Jméno a příjmení:   | <b>Maksim Zabrodin</b>                                                      |
| Osobní číslo:       | <b>I21262</b>                                                               |
| Studijní program:   | <b>B0688A140009 Informační technologie</b>                                  |
| Téma práce:         | <b>Aplikace pro zlepšení finanční gramotnosti a správu osobních financí</b> |
| Zadávající katedra: | <b>Katedra informačních technologií</b>                                     |

## Zásady pro vypracování

Cílem práce je vyvinout funkční desktopovou aplikaci pro zlepšení finanční gramotnosti a správu osobních financí uživatele. Součástí práce je také analýza a výběr vhodných nástrojů a technologií pro realizaci výstupní aplikace a odůvodnění jejich výběru. Aplikace pro finanční zdraví bude minimálně zahrnovat funkce pro analýzu finančního chování, tvorbu rozpočtu a finančního plánování do budoucna. Součástí aplikace budou také doporučení z oblasti finančního plánování závislé na chování uživatele. Aplikace bude splňovat bezpečnostní zásady spojené s tímto typem aplikace, které autor řádně analyzuje a popisuje.

Rozsah pracovní zprávy: **min. 30 s.,dop. rozsah 40 s.**  
Rozsah grafických prací:  
Forma zpracování bakalářské práce: **tištěná**

Seznam doporučené literatury:

GUPTA Arun. Building End-to-End Apps with C# 11 and .NET 7, 2023. Bpb Publications. ISBN 978-9355516374  
NORTHWOOD Chris. The Full Stack Developer, 2018. Apress. ISBN 978-1484241523

Vedoucí bakalářské práce: **Ing. Monika Borkovcová, Ph.D.**  
Katedra informačních technologií

Datum zadání bakalářské práce: **15. prosince 2024**  
Termín odevzdání bakalářské práce: **16. května 2025**

**prof. Ing. Petr Doležel, Ph.D.** v.r.  
děkan

L.S.

**Ing. Jan Panuš, Ph.D.** v.r.  
vedoucí katedry

V Pardubicích dne 28. února 2025

Prohlašuji:

Práci s názvem Aplikace pro zlepšení finanční gramotnosti a správu osobních financí jsem vypracoval samostatně a uvedl v ní všechny použité literární a informační zdroje v souladu s právními předpisy, vnitřními předpisy a vnitřními normami Univerzity Pardubice.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 7/2025 Pravidla pro odevzdávání, zpřístupňování závěrečných prací veřejnosti a jejich formální úpravu, bude závěrečná práce zpřístupněna veřejnosti prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 13. 05. 2026

Maksim Zabrodin

## **PODĚKOVÁNÍ**

Mé poděkování patří vedoucí bakalářské práce Ing. Monice Borkovcové, Ph.D., za odborné vedení, cenné rady a připomínky při zpracování této práce. Děkuji také své rodině a přátelům za podporu po celou dobu studia

## **ANOTACE**

Bakalářská práce se zabývá návrhem a implementací desktopové aplikace My Money, určené pro správu osobních a rodinných financí a podporu finanční gramotnosti uživatele. V teoretické části práce jsou popsány vybrané technologie a architektonické principy použité při vývoji aplikace, zejména platforma .NET, jazyk C#, technologie WPF, architektonický vzor MVVM, Dependency Injection, servisní a repozitářová vrstva a databáze SQLite. Praktická část práce se věnuje analýze požadavků, návrhu architektury, datového modelu a uživatelského rozhraní, popisu implementace a ověření výsledného řešení. Výstupem práce je funkční víceuživatelská desktopová aplikace s household modelem, rolmi uživatelů, správou osobních i sdílených financí, cíli spoření, analytickým dashboardem a modulem vysvětlitelné finanční analýzy.

## **KLÍČOVÁ SLOVA**

desktopová aplikace, finanční gramotnost, správa osobních financí, rodinné finance, household, C#, WPF, SQLite, MVVM.

## **TITLE**

Apps to Improve Financial Literacy and Manage Personal Finances

## **ANNOTATION**

This bachelor thesis focuses on the design and implementation of the My Money desktop application intended for personal and family finance management and support of users' financial literacy. The theoretical part describes the selected technologies and architectural principles used during development, especially the .NET platform, the C# programming language, WPF, the MVVM architectural pattern, dependency injection, the service and repository layers, and the SQLite database. The practical part deals with requirements analysis, architecture, data model and user interface design, implementation, and verification of the resulting solution. The output of the thesis is a functional multi-user desktop application with a household model, user roles, management of personal and shared finances, savings goals, an analytical dashboard, and an explainable financial analysis module.

## **KEYWORDS**

desktop application, financial literacy, personal finance management, family finances, household, C#, WPF, SQLite, MVVM.

# OBSAH

|                                                             |    |
|-------------------------------------------------------------|----|
| Obsah .....                                                 | 14 |
| Seznam obrázků .....                                        | 16 |
| Seznam tabulek .....                                        | 18 |
| Seznam zkratk .....                                         | 19 |
| Úvod .....                                                  | 13 |
| 1 Platforma .NET a jazyk C# .....                           | 14 |
| 1.1 Historie platformy .NET .....                           | 14 |
| 1.2 Windows Presentation Foundation a XAML .....            | 15 |
| 1.3 Architektonický vzor MVVM .....                         | 16 |
| 1.4 Datová vazba a ICommand ve WPF .....                    | 17 |
| 1.5 Dependency Injection .....                              | 18 |
| 1.6 Repository pattern a servisní vrstva .....              | 19 |
| 1.7 SQLite jako embedded databáze .....                     | 20 |
| 2 Analýza podobných řešení .....                            | 21 |
| 2.1 Aplikace zaměřené na správu osobních financí .....      | 21 |
| 2.2 Aplikace podporující sdílené nebo rodinné finance ..... | 21 |
| 2.3 Shrnutí a vymezení vlastního řešení .....               | 22 |
| 3 Analýza .....                                             | 23 |
| 3.1 Představení aplikace My Money .....                     | 23 |
| 3.2 Analýza požadavků .....                                 | 24 |
| 3.2.1 Funkční požadavky .....                               | 25 |
| 3.2.2 Nefunkční požadavky .....                             | 27 |
| 3.2.3 Bezpečnostní požadavky .....                          | 28 |
| 3.3 Případy užití .....                                     | 30 |
| 4 Návrh .....                                               | 32 |
| 4.1 Návrh architektury .....                                | 32 |
| 4.2 Datový model .....                                      | 34 |
| 4.3 Návrh uživatelského rozhraní .....                      | 35 |
| 4.4 Návrh navigace a toku dat .....                         | 44 |



|     |                                                     |    |
|-----|-----------------------------------------------------|----|
| 5   | Implementace .....                                  | 46 |
| 5.1 | Použité nástroje a technologie vývoje.....          | 46 |
| 5.2 | Struktura projektu.....                             | 47 |
| 5.3 | Autentizace a uživatelská relace .....              | 49 |
| 5.4 | Role a přístupová práva.....                        | 51 |
| 5.5 | Repositories a Services .....                       | 52 |
| 5.6 | Dashboard, analytika a Financial Health Score ..... | 53 |
| 5.7 | Transakce a konzistence dat.....                    | 56 |
| 6   | Testování .....                                     | 58 |
| 6.1 | Metody testování.....                               | 58 |
| 7   | Zhodnocení výsledků.....                            | 60 |
| 7.1 | Splnění cílů.....                                   | 60 |
| 7.2 | Omezení řešení.....                                 | 61 |
| 7.3 | Možnosti dalšího rozvoje .....                      | 61 |
|     | Závěr .....                                         | 63 |
|     | Přílohy.....                                        | 67 |

## SEZNAM OBRÁZKŮ

|                                                                                                                                                                 |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| <b>Obrázek 1:</b> Model-View-ViewModel pattern .....                                                                                                            | 16 |
| <b>Obrázek 2:</b> Vazba přes ICommand mezi Invokerem, Delegate Command a Receiverem .....                                                                       | 18 |
| <b>Obrázek 3:</b> Use case diagram aplikace My Money se znázorněním aktérů Guest, Admin, Partner a Child a hlavních případů užití systému (zdroj: vlastní)..... | 31 |
| <b>Obrázek 4:</b> Architektonické schéma aplikace My Money s rozdělením na View, ViewModel, Services, Repositories a SQLite databázi (zdroj: vlastní) .....     | 33 |
| <b>Obrázek 5:</b> ER diagram databázového modelu aplikace My Money (zdroj: vlastní) .....                                                                       | 35 |
| <b>Obrázek 6:</b> Přihlašovací obrazovka aplikace My Money (zdroj: vlastní).....                                                                                | 37 |
| <b>Obrázek 7:</b> Registrační obrazovka pro vytvoření prvního administrátorského účtu a household (zdroj: vlastní) .....                                        | 37 |
| <b>Obrázek 8:</b> Hlavní rozhraní aplikace po přihlášení uživatele a dokončení povinné změny hesla (zdroj: vlastní).....                                        | 38 |
| <b>Obrázek 9:</b> Dashboard s grafy Spending mix a Budget pressure (zdroj: vlastní) .....                                                                       | 39 |
| <b>Obrázek 10:</b> Dashboard s grafem Monthly trend a sekci Analysis engine s přehledem analytických metrik a doporučení (zdroj: vlastní) .....                 | 39 |
| <b>Obrázek 11:</b> Pokračování sekce Analysis engine s výstupy typu Observation a Praise (zdroj: vlastní) .....                                                 | 40 |
| <b>Obrázek 12:</b> Formulář pro přidání finančního záznamu (zdroj: vlastní).....                                                                                | 41 |
| <b>Obrázek 13:</b> Obrazovka historie finančních záznamů (zdroj: vlastní).....                                                                                  | 41 |
| <b>Obrázek 14:</b> Obrazovka pro správu budget categories a plánovaných částek (zdroj: vlastní) .....                                                           | 42 |
| <b>Obrázek 15:</b> Obrazovka savings goals s přehledem cílů a jejich plnění (zdroj: vlastní) ...                                                                | 42 |
| <b>Obrázek 16:</b> Obrazovka Household Members s tabulkou členů, rolí a oprávněními (zdroj: vlastní).....                                                       | 43 |
| <b>Obrázek 17:</b> Grafické porovnání členů household podle Financial Health Score v části Household Members (zdroj: vlastní) .....                             | 43 |
| <b>Obrázek 18:</b> Příklad postranní navigace pro child účet s omezenou sadou dostupných funkcí (zdroj: vlastní).....                                           | 44 |
| <b>Obrázek 19:</b> Struktura solution aplikace My Money v prostředí Visual Studio (zdroj: vlastní) .....                                                        | 49 |
| <b>Obrázek 20:</b> Ukázka e-mailové zprávy s dočasným heslem při vytvoření nového uživatelského účtu (zdroj: vlastní).....                                      | 50 |

|                                                                                                                                                                  |    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| <b>Obrázek 21:</b> Ukázka e-mailové zprávy s dočasným heslem při obnovení přístupu do aplikace (zdroj: vlastní) .....                                            | 50 |
| <b>Obrázek 22:</b> Obrazovka povinné změny hesla po přihlášení s dočasným heslem; navigační funkce jsou do dokončení změny hesla neaktivní (zdroj: vlastní)..... | 51 |
| <b>Obrázek 23:</b> Dashboard v personal kontextu se základním finančním přehledem a blokem Budget pulse (zdroj: vlastní).....                                    | 54 |
| <b>Obrázek 24:</b> Sekce Analysis engine v personal kontextu (zdroj: vlastní).....                                                                               | 55 |

## SEZNAM TABULEK

|                                                                 |    |
|-----------------------------------------------------------------|----|
| <b>Tabulka 1:</b> Funkční požadavky (zdroj: vlastní).....       | 25 |
| <b>Tabulka 2:</b> Nefunkční požadavky (zdroj: vlastní).....     | 27 |
| <b>Tabulka 3:</b> Bezpečnostní požadavky (zdroj: vlastní) ..... | 29 |

## SEZNAM ZKRATEK

|      |                                        |
|------|----------------------------------------|
| API  | Application Programming Interface      |
| DI   | Dependency Injection                   |
| ER   | Entity Relationship                    |
| KPI  | Key Performance Indicator              |
| MVVM | Model-View-ViewModel                   |
| SQL  | Structured Query Language              |
| UI   | User Interface                         |
| UML  | Unified Modeling Language              |
| UX   | User Experience                        |
| WPF  | Windows Presentation Foundation        |
| XAML | Extensible Application Markup Language |

# ÚVOD

Správa osobních financí patří mezi běžné součásti každodenního života. Přestože dnes existuje řada digitálních nástrojů pro sledování příjmů, výdajů a rozpočtu, ne všechny dokážou pokrýt potřeby uživatelů, kteří neřeší pouze vlastní finance, ale také finance sdílené v rámci domácnosti. V praxi se často objevuje potřeba oddělit osobní a společné výdaje, rozlišit oprávnění jednotlivých členů domácnosti a současně zachovat přehled nad celkovou finanční situací. Vedle samotné evidence financí je přitom důležitá i schopnost tato data správně vyhodnocovat a využívat je při běžném rozhodování.

Tato bakalářská práce se proto zabývá návrhem a implementací desktopové aplikace My Money, určené pro správu osobních a rodinných financí a pro podporu finanční gramotnosti uživatele. Cílem práce je vytvořit aplikaci, která umožní evidenci finančních záznamů, správu rozpočtových kategorií, sledování zůstatku a úspor, práci s cíli spoření a současně nabídne i víceuživatelské prostředí založené na modelu household. Součástí řešení je také rozlišení uživatelských rolí, práce s přístupovými právy a zobrazení přehledů, které uživateli pomohou lépe porozumět jeho finanční situaci.

Při zpracování práce bylo nejprve nutné seznámit se s vybranými technologiemi a návrhovými principy vhodnými pro tvorbu desktopové aplikace na platformě .NET. Následně byla provedena analýza podobných řešení a analýza požadavků, na jejichž základě byl vytvořen návrh architektury, datového modelu, uživatelského rozhraní a toku dat v aplikaci. Další část práce je věnována samotné implementaci navrženého řešení a ověření vybraných částí systému pomocí testování.

Práce je rozdělena na teoretickou a praktickou část. Teoretická část se zaměřuje na technologie a principy použité při tvorbě aplikace, zejména na platformu .NET, technologii WPF, architektonický vzor MVVM, datovou vazbu, Dependency Injection, repozitářovou a servisní vrstvu a databázi SQLite. Praktická část se věnuje analýze podobných řešení, analýze požadavků, návrhu aplikace, implementaci hlavních částí systému, testování a závěrečnému zhodnocení výsledného řešení.

# 1 PLATFORMA .NET A JAZYK C#

## 1.1 Historie platformy .NET

Platforma .NET vznikla jako iniciativa společnosti Microsoft s cílem vytvořit jednotné prostředí pro tvorbu různých typů aplikací. Na přelomu tisíciletí rostla potřeba propojit práci s desktopovými programy, webovými aplikacemi i serverovou částí systémů v rámci jednoho technologického základu. Microsoft na tuto situaci reagoval představením platformy, která nabídla společné běhové prostředí, rozsáhlé knihovny a podporu více programovacích jazyků. [1] [2]

Původní podoba platformy byla známá jako .NET Framework a byla úzce spojena s operačním systémem Windows. Její význam spočíval hlavně v tom, že sjednotila způsob práce s aplikační logikou, pamětí, systémovými službami i knihovnami tříd. Programátor tak měl k dispozici prostředí, ve kterém bylo možné vytvářet aplikace přehledněji a s menším důrazem na nízkoúrovňové technické detaily. [1]

S platformou .NET je od počátku spojen také jazyk C#. Ten byl navržen jako moderní objektově orientovaný jazyk s důrazem na srozumitelnou syntaxi, typovou bezpečnost a široké možnosti použití. Postupně se stal jedním z hlavních jazyků platformy a dnes se používá při tvorbě desktopových, webových i serverových aplikací. Jeho rozšíření významně přispělo k tomu, že se .NET stal jedním z nejpoužívanějších technologických základů v prostředí Microsoftu. [3] [2]

Později začalo být zřejmé, že původní podoba platformy neodpovídá všem novým požadavkům. Stále větší důraz byl kladen na otevřenost, modularitu, vyšší výkon a podporu více operačních systémů. Na tuto změnu navázal vznik .NET Core, který přinesl odlišný přístup k celé platformě a otevřel ji širšímu použití i mimo samotné prostředí Windows. Následně došlo ke sjednocení moderních větví pod společné označení .NET, které dnes představuje obecný základ pro tvorbu širokého spektra aplikací. [1] [2]

Z historického hlediska je .NET příkladem platformy, která prošla výraznou proměnou. Z původně spíše uzavřeného řešení orientovaného hlavně na Windows se stalo univerzálnější prostředí použitelné v mnoha oblastech současného programování. Právě tato schopnost postupné proměny a přizpůsobení novým požadavkům patří mezi důvody, proč si platforma .NET udržela význam i v současné softwarové praxi. [2]

## 1.2 Windows Presentation Foundation a XAML

Při tvorbě desktopových aplikací pro operační systém Windows hraje důležitou roli volba technologie určené pro návrh uživatelského rozhraní. V prostředí platformy .NET patří mezi nejznámější technologie tohoto typu Windows Presentation Foundation (WPF). Jde o framework určený pro tvorbu grafických desktopových aplikací, který umožňuje vytvářet okna, ovládací prvky, formuláře, tabulky i další části rozhraní v rámci jednoho uceleného systému. [4]

Jednou z podstatných vlastností WPF je oddělení vizuální podoby aplikace od programové logiky. To je důležité hlavně u rozsáhlejších projektů, kde je potřeba udržet kód přehledný a jednotlivé části systému od sebe co nejvíce oddělit. WPF současně nabízí i širší možnosti práce se styly, šablonami, zdroji, animacemi a datovou vazbou, proto se hodí nejen pro jednoduchá okna, ale i pro složitější desktopové aplikace. [5]

S technologií WPF úzce souvisí jazyk XAML (eXtensible Application Markup Language). Jde o značkovací jazyk, pomocí kterého se popisuje struktura a vzhled uživatelského rozhraní. V XAML lze definovat rozmístění prvků, jejich vlastnosti i vazby na data. Tento způsob zápisu zjednodušuje práci s vizuální částí aplikace a přispívá k tomu, že rozhraní není nutné vytvářet celé pouze v programovém kódu. [6]

Význam XAML spočívá i v tom, že umožňuje přehledně oddělit rozhraní od aplikační logiky. Grafická část systému tak může být spravována samostatně, zatímco logika zůstává v odpovídajících třídách aplikace. Tento přístup je praktický hlavně tam, kde je potřeba obrazovky průběžně upravovat, doplňovat nové prvky nebo měnit jejich uspořádání bez zásadního zásahu do vnitřního fungování programu. [7]

Další důležitou vlastností WPF je podpora datově orientovaného rozhraní. Jednotlivé prvky mohou být navázány na objekty a jejich vlastnosti, takže změny dat se mohou automaticky promítat do zobrazení. To je užitečné například u formulářů, seznamů nebo přehledových obrazovek, kde je potřeba pracovat s větším množstvím měnících se údajů. [6] [8]

WPF a XAML tak tvoří důležitý základ pro tvorbu moderních desktopových aplikací v prostředí Windows. Jejich význam nespočívá jen v samotném vytvoření grafického rozhraní, ale i v tom, že podporují přehlednější organizaci projektu, práci s daty a oddělení jednotlivých částí systému. [4] [7]



### 1.3 Architektonický vzor MVVM

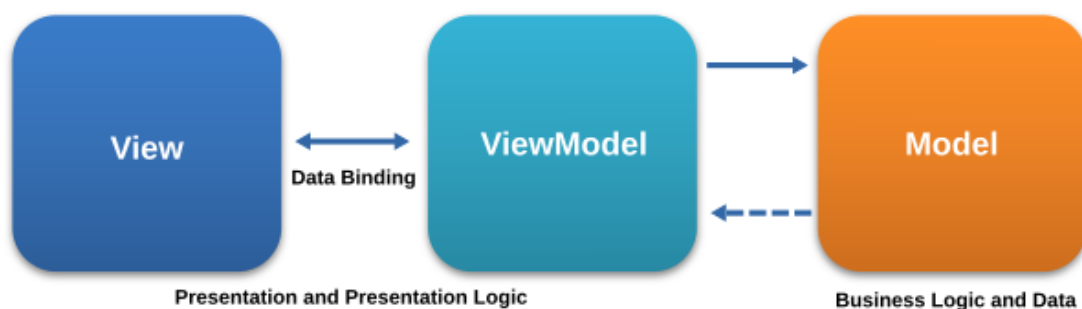
Při návrhu desktopových aplikací nestačí zvolit pouze vhodné technologie, ale také způsob, jak bude celý projekt strukturován. V prostředí WPF se velmi často používá architektonický vzor MVVM (Model-View-ViewModel). Tento přístup slouží k oddělení uživatelského rozhraní, aplikační logiky a práce s daty, a tím přispívá k přehlednější organizaci programu. [9]

Základní myšlenkou MVVM je rozdělení aplikace do tří částí. Model představuje datovou a doménovou vrstvu, tedy objekty a pravidla, se kterými systém pracuje. View tvoří samotné uživatelské rozhraní a zajišťuje zobrazení dat. ViewModel stojí mezi nimi a propojuje vizuální část aplikace s daty a logikou systému. [6]

Model obsahuje především datové struktury a objekty, které reprezentují hlavní prvky dané oblasti. Neměl by být závislý na konkrétní podobě uživatelského rozhraní, protože jeho úkolem je popsat samotná data a jejich význam v systému. [7]

View představuje obrazovky, formuláře, tabulky, tlačítka a další prvky, se kterými uživatel přímo pracuje. V ideálním případě má tato vrstva sloužit hlavně k prezentaci dat a předávání uživatelských akcí dále do systému. V prostředí WPF bývá View obvykle vytvářena pomocí jazyka XAML, proto je její struktura dobře čitelná a oddělená od vnitřní logiky aplikace. [9]

Nejdůležitější roli v tomto vzoru má ViewModel. Obsahuje vlastnosti a příkazy, které jsou navázány na prvky uživatelského rozhraní. Přijímá data z Modelu, upravuje je do podoby vhodné pro zobrazení a reaguje na akce uživatele. Tím se omezuje množství logiky přímo v View a celý projekt zůstává lépe organizovaný. [9]



**Obrázek 1:** Model-View-ViewModel pattern (zdroj: wikipedia.org [online]. Dostupný na WWW: <https://commons.wikimedia.org/w/index.php?curid=154009347>)

Výhodou architektury MVVM je hlavně lepší přehlednost a snazší údržba kódu. Pokud jsou rozhraní, datový model a prezentační logika správně odděleny, je jednodušší projekt

rozšiřovat, upravovat i testovat. Změny ve vzhledu obrazovek tak mají menší dopad na logiku aplikace a naopak. [7] [6]

Architektonický vzor MVVM je proto považován za vhodné řešení zejména pro desktopové aplikace vytvářené ve WPF, kde dobře odpovídá principu oddělení zodpovědností mezi jednotlivými částmi systému. [9]

## **1.4 Datová vazba a ICommand ve WPF**

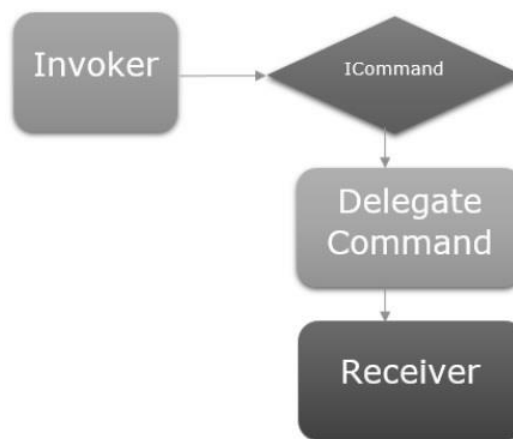
Jednou z důležitých vlastností technologie WPF je vestavěný mechanismus datové vazby (data binding). Ten umožňuje propojit uživatelské rozhraní s daty a aplikační logikou bez potřeby ručně aktualizovat jednotlivé prvky přímo v programovém kódu. Ve spojení s architektonickým vzorem MVVM jde o jeden ze základních prostředků pro tvorbu přehledných desktopových aplikací. [10] [6]

Datová vazba ve WPF znamená, že vlastnost vizuálního prvku je propojena s vlastností datového objektu. Změna hodnoty ve zdroji dat se pak může automaticky projevit v uživatelském rozhraní, případně i opačně. Tento princip zjednodušuje práci s formuláři, seznamy, tabulkami i přehledovými obrazovkami a omezuje množství pomocného kódu. [10] [8]

V architektuře MVVM bývá zdrojem dat pro View nejčastěji ViewModel. Ten poskytuje vlastnosti, které jsou následně navázány na jednotlivé prvky rozhraní. Uživatelské rozhraní tak získává data v podobě vhodné pro zobrazení, zatímco logika jejich získávání a změn zůstává oddělena ve vnitřní části aplikace. [10] [7]

Vedle datové vazby hraje ve WPF důležitou roli také mechanismus ICommand. Toto rozhraní slouží k předávání akcí uživatele z View do ViewModelu bez potřeby vkládat obsluhu událostí přímo do code-behind. Typickým příkladem je situace, kdy je stisk tlačítka svázan s příkazem, který provede určitou operaci v logice aplikace. [11]

Výhodou příkazového mechanismu je i to, že neřeší pouze samotné spuštění akce, ale může určovat také to, zda je daná operace v konkrétním stavu aplikace vůbec dostupná. Uživatelské rozhraní tak může reagovat na změny stavu dat a podle potřeby zpřístupňovat nebo blokovat určité akce. [11]



**Obrázek 2:** Vazba přes ICommand mezi Invokerem, Delegate Command a Receiverem (zdroj: TutorialsPoint [online]. Dostupné na WWW:

[https://www.tutorialspoint.com/mvvm/mvvm\\_view\\_viewmodel\\_communication.htm](https://www.tutorialspoint.com/mvvm/mvvm_view_viewmodel_communication.htm))

Datová vazba a ICommand tak ve WPF významně podporují oddělení rozhraní od logiky aplikace. Společně s architekturou MVVM přispívají k tomu, že projekt zůstává lépe organizovaný, přehlednější a snáze udržitelný. [12]

## 1.5 Dependency Injection

Při návrhu softwarových aplikací je důležité usilovat o takovou strukturu programu, která bude přehledná, rozšiřitelná a snadno udržitelná. Jedním z přístupů, který k tomu přispívá, je Dependency Injection (DI). Tento princip vychází z myšlenky, že objekt nemá své závislosti vytvářet sám, ale má je získat z vnějšího prostředí. Tím se snižuje provázanost mezi jednotlivými částmi systému a zlepšuje se architektura aplikace. [6] [7]

V tradičně navrženém programu třída často přímo vytváří instance dalších tříd, které potřebuje ke své činnosti. Takové řešení může být zpočátku jednoduché, avšak při růstu projektu vede ke komplikacím. Jednotlivé komponenty se pevně vážou na konkrétní implementace, a tím se ztěžují změny, rozšíření i testování systému. [7]

Princip Dependency Injection tento problém řeší tím, že závislosti jsou objektu předávány zvenčí, nejčastěji prostřednictvím konstruktoru. Třída tak pouze deklaruje, co ke své činnosti potřebuje, ale sama nerozhoduje o tom, jak budou konkrétní závislosti vytvořeny. Tím se odděluje odpovědnost za logiku třídy od odpovědnosti za sestavení a konfiguraci objektů v aplikaci. [13]

Významnou roli zde hrají také rozhraní. Pokud třída závisí na abstrakci namísto konkrétní implementace, je možné jednotlivé části systému snadněji měnit nebo nahrazovat bez zásahu

do vyšších vrstev aplikace. Tento přístup odpovídá zásadám objektově orientovaného návrhu, zejména snaze o nižší provázanost a vyšší modularitu programu. [13]

V prostředí platformy .NET je Dependency Injection běžně podporován prostřednictvím kontejneru služeb, který slouží k registraci a následnému poskytování jednotlivých závislostí. Při spuštění aplikace je možné určit, jaké služby budou v systému dostupné, jaké rozhraní odpovídá které implementaci a jaký životní cyklus budou mít jednotlivé objekty. Tím lze centrálně řídit sestavení aplikace a udržet její strukturu konzistentní. [14]

Použití Dependency Injection přináší několik důležitých výhod. Zlepšuje přehlednost architektury, usnadňuje rozšiřování aplikace a podporuje testovatelnost systému, protože při ověřování jednotlivých částí programu lze snadněji nahradit reálné závislosti zjednodušenými nebo testovacími implementacemi. Dependency Injection je proto vhodný zejména pro vícevrstvé aplikace, ve kterých spolupracuje prezentační vrstva, aplikační logika a vrstva přístupu k datům. [6] [13]

## **1.6 Repository pattern a servisní vrstva**

Při tvorbě aplikací, které pracují s daty, je důležité rozdělit jednotlivé odpovědnosti mezi více částí systému. Pokud se na jednom místě mísí práce s databází, aplikační logika i reakce na uživatelské akce, začne být kód rychle nepřehledný a hůře se udržuje. Z tohoto důvodu se v praxi často používá Repository pattern a servisní vrstva. [13] [7]

Repository pattern slouží k oddělení přístupu k datům od ostatních částí aplikace. Repozitář představuje vrstvu, která zajišťuje načítání, ukládání, úpravy a mazání dat. Ostatní části systému pak nemusí přímo pracovat s SQL dotazy, databázovým připojením nebo konkrétním způsobem ukládání dat, ale komunikují pouze s rozhraním repozitáře. [13] [15]

Samotný repozitář ale obvykle nestačí pro zachycení celé logiky aplikace. Většina systémů obsahuje operace, které zahrnují více kroků, kombinují data z různých míst nebo kontrolují určitá pravidla. Právě pro tyto situace se používá servisní vrstva. Ta stojí nad repozitáři a soustřeďuje aplikační nebo obchodní logiku systému. [13]

Servisní vrstva tak může využívat jeden nebo více repozitářů a nad nimi provádět složitější operace. Může například ověřovat oprávnění, řídit pořadí jednotlivých kroků nebo připravovat data pro další část aplikace. Výhodou tohoto přístupu je, že logika není rozptýlena po různých třídách, ale zůstává soustředěna na místě, kde se s ní lépe pracuje a kde se také snáze upravuje. [16]

Kombinace repozitářů a servisní vrstvy je vhodná hlavně u vícevrstvých aplikací. Repozitáře řeší technickou stránku práce s daty, zatímco servisní vrstva určuje, jak budou tato data použita v logice systému. Takové rozdělení zlepšuje přehlednost projektu, podporuje modularitu a usnadňuje pozdější rozšiřování i testování aplikace. [7] [16]

## **1.7 SQLite jako embedded databáze**

Při návrhu menších desktopových aplikací je důležité zvolit vhodný způsob ukládání dat. Ne každé řešení vyžaduje plnohodnotný databázový server. U aplikací, které jsou určeny hlavně pro lokální použití a nepracují s velkým počtem současně připojených uživatelů, bývá vhodnější jednodušší přístup. Jednou z nejznámějších možností tohoto typu je SQLite. [17] [18]

SQLite je lehká relační databáze, která ukládá data obvykle do jednoho souboru. Na rozdíl od serverových databázových systémů nevyžaduje samostatný serverový proces, složitou konfiguraci ani správu připojení přes síť. Právě proto bývá často používána u aplikací, kde je důležitá jednoduchá instalace, přenositelnost a nenáročný provoz. [17] [19]

Výhodou SQLite je i to, že i přes svou jednoduchost podporuje běžné prvky relační databáze. Umožňuje vytvářet tabulky, používat SQL dotazy, indexy i omezení integrity dat. Pro řadu menších informačních systémů nebo desktopových aplikací je takové řešení plně dostačující. [19]

Dobře se uplatňuje hlavně tam, kde mají data menší objem a kde není potřeba budovat složitější databázovou infrastrukturu. Lokální databázový soubor navíc umožňuje, aby aplikace fungovala samostatně i bez připojení k internetu nebo bez závislosti na externím serveru. [17]

V prostředí .NET lze SQLite využívat prostřednictvím odpovídajících knihoven a rozhraní ADO.NET. To umožňuje provádět SQL dotazy, načítat a ukládat data a začlenit práci s databází do běžné struktury aplikace. I z tohoto důvodu patří SQLite mezi časté volby při tvorbě menších desktopových řešení. [18]

SQLite tedy představuje vhodnou databázovou technologii pro aplikace, u kterých je důležité jednoduché, lokální a spolehlivé ukládání dat bez potřeby serverové infrastruktury. [18]

## 2 ANALÝZA PODOBNÝCH ŘEŠENÍ

Oblast správy osobních financí je dnes poměrně dobře pokryta řadou hotových aplikací. Většina z nich se zaměřuje na evidenci příjmů a výdajů, tvorbu rozpočtů, sledování úspor a zobrazení základních přehledů. Častým prvkem bývá také synchronizace s bankovními účty, mobilní přístup a práce s grafickými statistikami. Vedle jednoduchých osobních nástrojů už existují i řešení, která podporují sdílené finance nebo spolupráci více uživatelů. Právě z tohoto důvodu je vhodné stručně porovnat podobná řešení a vymezit, v čem se navrhovaná aplikace liší. [20] [21]

### 2.1 Aplikace zaměřené na správu osobních financí

Mezi známá řešení patří například YNAB nebo Wallet od společnosti BudgetBakers. YNAB je postaven především na plánování rozpočtu, práci s cíli a pravidelném sledování výdajů. Vedle běžné osobní správy financí nabízí také sdílení v rámci skupiny YNAB Together, kde může více uživatelů pracovat s jedním předplatným, sdílet vybrané rozpočty a používat samostatné přihlášení. Součástí aplikace jsou i přehledy, sledování cílů a synchronizace mezi více zařízeními. [20]

Aplikace Wallet se zaměřuje na přehled osobních a rodinných financí, rozpočty, sledování výdajů, plánované platby a bankovní synchronizaci. Zajímavou částí je i funkce Group Sharing, která umožňuje přepínat mezi osobním a sdíleným prostorem a přiřazovat členům různé úrovně přístupu, například Admin access, Track and read nebo Read only. Toto řešení je v některých ohledech blízké víceuživatelskému modelu, stále je však navázáno hlavně na cloudové a mobilní použití. [21]

### 2.2 Aplikace podporující sdílené nebo rodinné finance

Další skupinu tvoří aplikace, které se více zaměřují na sdílené finance. Spendee nabízí funkci Shared Wallets, určenou pro partnery, rodiny nebo jiné skupiny uživatelů. Uživatelé mohou společně sledovat výdaje a mít přehled o tom, kdo platil a kolik bylo celkem utraceno. Zároveň má však toto řešení i určitá omezení. Podle oficiální dokumentace nelze sdílet bankovní účty v podobě bank account wallets a sdílené rozpočty nejsou podporovány jako plnohodnotná společná funkcionalita. [22]

Na podobném principu pracuje také Monarch Money, které je navrženo s ohledem na household použití. Uživatelé mají vlastní přihlášení, ale pracují nad jedním společným dashboardem, sdíleným rozpočtem a společným pohledem na finance domácnosti. Toto řešení dobře podporuje společný přehled nad financemi, podle oficiálního popisu však

členové household pracují se stejnou úrovní viditelnosti a přístupu. Model rolí je zde tedy jednodušší než v systému, který rozlišuje různé typy členů domácnosti a jemnější oprávnění. [23]

## **2.3 Shrnutí a vymezení vlastního řešení**

Z porovnání vyplývá, že současné aplikace pro správu financí obvykle velmi dobře pokrývají osobní rozpočtování, sledování výdajů, grafické přehledy a v řadě případů i synchronizaci s bankovními účty. Část z nich již podporuje i sdílené finance, společné dashboardy nebo sdílené peněženky. Méně často se však objevuje kombinace více oddělených finančních kontextů, jemnější správy rolí v rámci household a přímé vazby mezi přístupovými právy, navigací a aplikační logikou. [20] [21] [23]

Navrhovaná aplikace My Money se od uvedených řešení liší především tím, že je postavena jako desktopová aplikace s lokální databází a víceuživatelským household modelem, který rozlišuje role Admin, Partner a Child. Součástí návrhu je oddělení personal a shared financí, správa přístupových práv, zvláštní práce s child účty a také analytická vrstva založená na Financial Health Score a modulu explainable analytics. Cílem tedy nebylo vytvořit jen další evidenční nástroj, ale navrhnout řešení, které propojí správu financí, víceuživatelské prostředí a srozumitelnou interpretaci finančních dat.

### 3 ANALÝZA

Tato část práce se věnuje analýze navrhovaného řešení z hlediska jeho zaměření a požadavků, které na něj byly kladeny. Nejprve je stručně představena aplikace My Money a následně jsou shrnuty funkční, nefunkční a bezpečnostní požadavky, které tvořily základ pro další návrh a implementaci systému.

#### 3.1 Představení aplikace My Money

Aplikace My Money představuje desktopovou aplikaci pro správu osobních a rodinných financí. Jejím hlavním cílem je poskytnout uživateli pohodlný nástroj pro evidenci příjmů a výdajů, kontrolu rozpočtu, plánování úspor a analýzu finančního chování. Na rozdíl od jednoduchého jednouživatelského systému evidence je aktuální verze aplikace určena nejen pro jednoho vlastníka dat, ale i pro využití v rámci společného rodinného finančního prostoru.

Klíčovou vlastností aplikace je podpora modelu household, v jehož rámci může několik uživatelů pracovat se společnými finančními daty podle své role a sady oprávnění. V systému jsou definovány role Admin, Partner a Child. Toto rozdělení umožňuje modelovat běžné rodinné scénáře použití, ve kterých mají různí účastníci odlišnou úroveň přístupu k datům a funkcím aplikace.

Aplikace podporuje registraci prvního uživatele se současným vytvořením household, přihlášení do systému, ukládání aktivní uživatelské relace a také obnovu přístupu pomocí e-mailu. Pro dětské účty je navržen samostatný scénář správy hesla, ve kterém je heslo nastavováno a měněno administrátorem. Tímto způsobem je v aplikaci realizován základní mechanismus autentizace a řízení přístupu pro lokální víceuživatelský desktopový systém.

Z hlediska domény aplikace pracuje s několika finančními kontexty. Uživatel může interagovat se společnými rodinnými daty (Household), s osobním finančním kontextem (Personal) a v případě dětského účtu se svou aktivitou v rámci společného rodinného prostoru. Toto rozdělení ovlivňuje práci s finančními záznamy, rozpočtovými kategoriemi, cíli úspor, analytikou i zobrazováním dat na dashboardu. Aplikace se tak neomezuje na jednoduché ukládání seznamu výdajů, ale podporuje širší strukturu finančních entit a uživatelských scénářů.

Funkčně My Money zahrnuje několik hlavních modulů. Systém umožňuje přidávat a zobrazovat finanční záznamy, spravovat rozpočtové kategorie, sledovat zůstatek a úspory, vytvářet cíle spoření a kontrolovat jejich plnění. Samostatná část aplikace je určena pro



správu členů household, včetně změny rolí a přístupových práv. Pro některé kategorie uživatelů platí dodatečná omezení. Například child účet nemá přístup k personal finance kontextu, nemůže mazat záznamy historie, nemůže spravovat složení rodiny a má omezený přístup ke změnám cílů úspor. Tato omezení jsou uplatňována jak na úrovni rozhraní, tak na úrovni business logiky.

Významně je rozšířena analytická část aplikace. Hlavní dashboard plní nejen informační, ale i analytickou funkci. Jsou v něm realizovány vizuální a výpočetní prvky, které umožňují hodnotit strukturu výdajů, úroveň využití rozpočtu a dynamiku finančního chování. Uživatel má k dispozici grafy rozložení výdajů podle kategorií, porovnání kategorií s rozpočtovými limity, měsíční trend výdajů a textovou interpretaci aktuálního stavu rozpočtu.

Samostatný význam má vestavěný modul explainable analytics, založený na systému formalizovaných pravidel. Na rozdíl od obecných doporučení tento modul identifikuje konkrétní finanční vzorce, vysvětluje jejich význam a vytváří pozorování, varování, doporučení nebo pozitivní hodnocení. V aplikaci je také realizován ukazatel Financial Health Score, který umožňuje hodnotit finanční disciplínu uživatele nebo člena household na škále od 0 do 100.

Z technického hlediska je aplikace realizována jako desktopová aplikace na platformě .NET s využitím jazyka C#, technologie WPF, architektonického přístupu MVVM, vestavěné lokální databáze SQLite a servisně-repozitářové organizace logiky. Pro operace ovlivňující finanční stav je realizována podpora transakcí, která pomáhá zachovat konzistenci dat při přidávání, mazání a úpravě záznamů. Kromě toho projekt obsahuje samostatný testovací modul se základními unit testy pro ověření analytické logiky.

My Money tak nepředstavuje pouze aplikaci pro evidenci výdajů, ale komplexnější household orientovaný systém pro správu financí. Kombinuje víceuživatelský model, řízení přístupových práv, práci s financemi v různých kontextech, vizualizaci dat, vysvětlitelnou analytiku a základní mechanismy spolehlivosti. Tím je aplikace nejen prakticky užitečná, ale také dostatečně komplexní z hlediska bakalářské práce zaměřené na návrh a realizaci desktopového softwarového řešení.

### **3.2 Analýza požadavků**

Před zahájením návrhu a implementace aplikace bylo nutné stanovit základní požadavky, které má výsledné řešení splňovat. Analýza požadavků představuje důležitou část návrhu systému, protože vymezuje, jaké funkce má aplikace poskytovat, jaké vlastnosti má mít z hlediska kvality a jaké zásady musí být dodrženy při práci s uživatelskými a finančními daty.

Vzhledem k tomu, že aplikace My Money byla navržena jako víceuživatelská desktopová aplikace pro správu osobních a rodinných financí, bylo nutné zohlednit také požadavky související s autorizací, rolemi uživatelů a přístupovými právy. V rámci této práce byly požadavky rozděleny na funkční, nefunkční a bezpečnostní. [24]

### 3.2.1 Funkční požadavky

Funkční požadavky určují, jaké činnosti musí aplikace My Money umožňovat svým uživatelům. Vzhledem k tomu, že se jedná o víceuživatelskou desktopovou aplikaci pro správu osobních a rodinných financí, zahrnují tyto požadavky nejen práci s finančními daty, ale také správu household, přístupových práv a analytických funkcí. [24]

**Tabulka 1:** Funkční požadavky (zdroj: vlastní)

|    |                           |                                                                                                                                   |
|----|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| F1 | Registrace household      | Aplikace bude umožňovat registraci prvního uživatele a současné vytvoření nového household.                                       |
| F2 | Přihlášení uživatele      | Aplikace bude umožňovat přihlášení uživatele pomocí e-mailu a hesla.                                                              |
| F3 | Správa uživatelské relace | Aplikace bude po přihlášení uchovávat aktuální uživatelskou relaci a pracovat v kontextu přihlášeného uživatele a jeho household. |
| F4 | Obnovení hesla            | Aplikace bude umožňovat obnovení hesla dospělého účtu prostřednictvím e-mailu s dočasným heslem.                                  |
| F5 | Správa child účtů         | Administrátor bude moci nastavovat a měnit hesla child účtů.                                                                      |
| F6 | Správa členů household    | Administrátor bude moci přidávat nové členy household, upravovat jejich údaje a odstraňovat je.                                   |
| F7 | Role a oprávnění          | Aplikace bude podporovat role Admin, Partner a Child a bude umožňovat nastavovat oprávnění CanManageBudget a CanManageMembers.    |
| F8 | Role-based přístup        | Aplikace bude zpřístupňovat nebo omezovat jednotlivé funkce podle role uživatele a jeho oprávnění.                                |

|     |                               |                                                                                                                                          |
|-----|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| F9  | Finanční kontexty             | Aplikace bude podporovat více finančních kontextů, zejména Household a Personal.                                                         |
| F10 | Správa finančních záznamů     | Aplikace bude umožňovat vytvářet finanční záznamy s uvedením částky, data, kategorie, typu a popisu.                                     |
| F11 | Historie záznamů              | Aplikace bude umožňovat zobrazovat historii finančních záznamů, filtrovat ji podle období a řadit podle vybraných kritérií.              |
| F12 | Mazání záznamů                | Aplikace bude umožňovat mazání finančních záznamů pouze uživatelům, kteří k tomu mají oprávnění.                                         |
| F13 | Správa rozpočtových kategorií | Aplikace bude umožňovat vytvářet, upravovat a odstraňovat rozpočtové kategorie v závislosti na zvoleném kontextu a oprávnění uživatele.  |
| F14 | Plánování rozpočtu            | U rozpočtových kategorií bude možné nastavovat plánované částky a porovnávat je se skutečnými výdaji.                                    |
| F15 | Správa zůstatku a úspor       | Aplikace bude umožňovat evidovat a průběžně aktualizovat zůstatek a úspory uživatele nebo household.                                     |
| F16 | Cíle spoření                  | Aplikace bude umožňovat vytvářet, upravovat a sledovat cíle spoření.                                                                     |
| F17 | Child pravidla pro goals      | Child účet bude moci vytvářet shared cíle, přispívat na rodinné cíle a upravovat pouze cíle, které sám vytvořil.                         |
| F18 | Dashboard                     | Aplikace bude zobrazovat přehled finanční situace uživatele nebo household, zejména výdaje, zůstatek, úspory a další souhrnné ukazatele. |
| F19 | Vizualizace dat               | Dashboard bude obsahovat grafické přehledy, zejména Spending mix, Budget pressure a Monthly trend.                                       |

|     |                           |                                                                                                                                     |
|-----|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| F20 | Explainable analytics     | Aplikace bude poskytovat analytické výstupy a doporučení založené na pravidlech, včetně typů Warning, Advice, Observation a Praise. |
| F21 | Financial Health Score    | Aplikace bude vypočítávat ukazatel Financial Health Score a zobrazovat jej u jednotlivých členů household.                          |
| F22 | Porovnání členů household | Aplikace bude umožňovat zobrazit srovnání členů household podle Financial Health Score.                                             |

### 3.2.2 Nefunkční požadavky

Nefunkční požadavky určují kvalitativní vlastnosti, které má aplikace splňovat. V případě aplikace My Money se jedná zejména o požadavky na přehlednost, spolehlivost, lokální provoz, udržitelnost, rozšiřitelnost a konzistentní chování systému při práci s finančními daty. [24]

**Tabulka 2:** Nefunkční požadavky (zdroj: vlastní)

|    |                       |                                                                                                                                                                                                           |
|----|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| N1 | Typ aplikace          | Aplikace bude realizována jako desktopová aplikace pro operační systém Windows.                                                                                                                           |
| N2 | Lokální provoz        | Hlavní funkce aplikace budou dostupné v lokálním prostředí bez nutnosti trvalého připojení k internetu; síťové připojení bude vyžadováno pouze u vybraných funkcí, například při obnovení hesla e-mailem. |
| N3 | Uživatelské rozhraní  | Uživatelské rozhraní bude přehledné, srozumitelné a vhodné pro běžného uživatele bez odborných technických znalostí.                                                                                      |
| N4 | Jednoduchost ovládání | Základní operace aplikace musí být dostupné bez složité navigace a bez nutnosti dlouhého zaškolení uživatele.                                                                                             |
| N5 | Přehlednost dat       | Finanční údaje, grafy a analytické výstupy budou zobrazeny způsobem, který uživateli umožní rychlou orientaci v aktuální finanční situaci.                                                                |

|     |                             |                                                                                                                                             |
|-----|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| N6  | Spolehlivost ukládání dat   | Aplikace bude uchovávat data stabilně a zajistí jejich dostupnost i po opakovaném ukončení a spuštění programu.                             |
| N7  | Konzistence operací         | Operace ovlivňující finanční stav systému budou prováděny konzistentně tak, aby nedocházelo k nesouladu mezi záznamy, zůstatkem a úsporami. |
| N8  | Architektonická přehlednost | Aplikace bude navržena s oddělením prezentační vrstvy, aplikační logiky a vrstvy přístupu k datům.                                          |
| N9  | Udržitelnost                | Struktura projektu bude organizována tak, aby bylo možné aplikaci dále upravovat a rozvíjet bez zásadního narušení stávajícího řešení.      |
| N10 | Rozšiřitelnost              | Návrh aplikace bude umožňovat budoucí doplnění dalších rolí, pravidel analýzy, grafických přehledů nebo nových finančních funkcí.           |
| N11 | Testovatelnost              | Aplikační logika bude navržena tak, aby bylo možné vybrané části systému ověřovat pomocí unit testů.                                        |
| N12 | Výkon                       | Aplikace bude dostatečně výkonná pro běžné použití v rámci menšího household a pro práci s obvyklým objemem finančních záznamů.             |
| N13 | Jednoduchost nasazení       | Řešení nebude vyžadovat složitou serverovou infrastrukturu ani komplikovanou instalaci databázového prostředí.                              |

### 3.2.3 Bezpečnostní požadavky

Bezpečnostní požadavky vymezují zásady, které musí aplikace dodržovat při práci s uživatelskými účty, přístupovými právy a finančními daty. V případě aplikace My Money se jedná zejména o ochranu přihlašovacích údajů, řízení přístupu podle rolí, bezpečné ukládání dat a omezení neoprávněných zásahů do sdílených finančních informací. [25]

**Tabulka 3:** Bezpečnostní požadavky (zdroj: vlastní)

|     |                                     |                                                                                                                                                                |
|-----|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| B1  | Autentizace uživatele               | Aplikace bude umožňovat přístup pouze ověřeným uživatelům přihlášeným pomocí platných přihlašovacích údajů.                                                    |
| B2  | Ochrana hesel                       | Hesla uživatelů nebudou ukládána v otevřené podobě, ale pouze ve formě hashované hodnoty.                                                                      |
| B3  | Obnovení přístupu                   | Aplikace bude umožňovat obnovení přístupu prostřednictvím dočasného hesla zasláného na e-mail dospělého uživatele.                                             |
| B4  | Vynucená změna hesla                | Po přihlášení pomocí dočasného hesla bude uživatel vyzván ke změně hesla před dalším používáním aplikace.                                                      |
| B5  | Role-based access control           | Přístup k funkcím aplikace bude řízen podle role uživatele a přiřazených oprávnění.                                                                            |
| B6  | Ochrana shared dat                  | Uživatel nebude moci provádět operace nad sdílenými daty household, pokud k tomu nemá odpovídající oprávnění.                                                  |
| B7  | Omezení child účtů                  | Child účet bude mít omezený přístup k vybraným funkcím aplikace, zejména ke správě členů household, mazání historie a správě některých rozpočtových kategorií. |
| B8  | Kontrola oprávnění v logice systému | Omezení přístupu nebudou realizována pouze na úrovni uživatelského rozhraní, ale také na úrovni aplikační logiky a služeb.                                     |
| B9  | Validace vstupů                     | Uživatelské vstupy budou kontrolovány z hlediska správnosti, úplnosti a přípustného formátu.                                                                   |
| B10 | Bezpečný přístup k databázi         | Komunikace s databází bude realizována řízeným způsobem prostřednictvím repozitářů a parametrizovaných SQL dotazů.                                             |
| B11 | Konzistence finančních operací      | Operace ovlivňující finanční stav aplikace budou prováděny transakčně, aby nedocházelo k                                                                       |

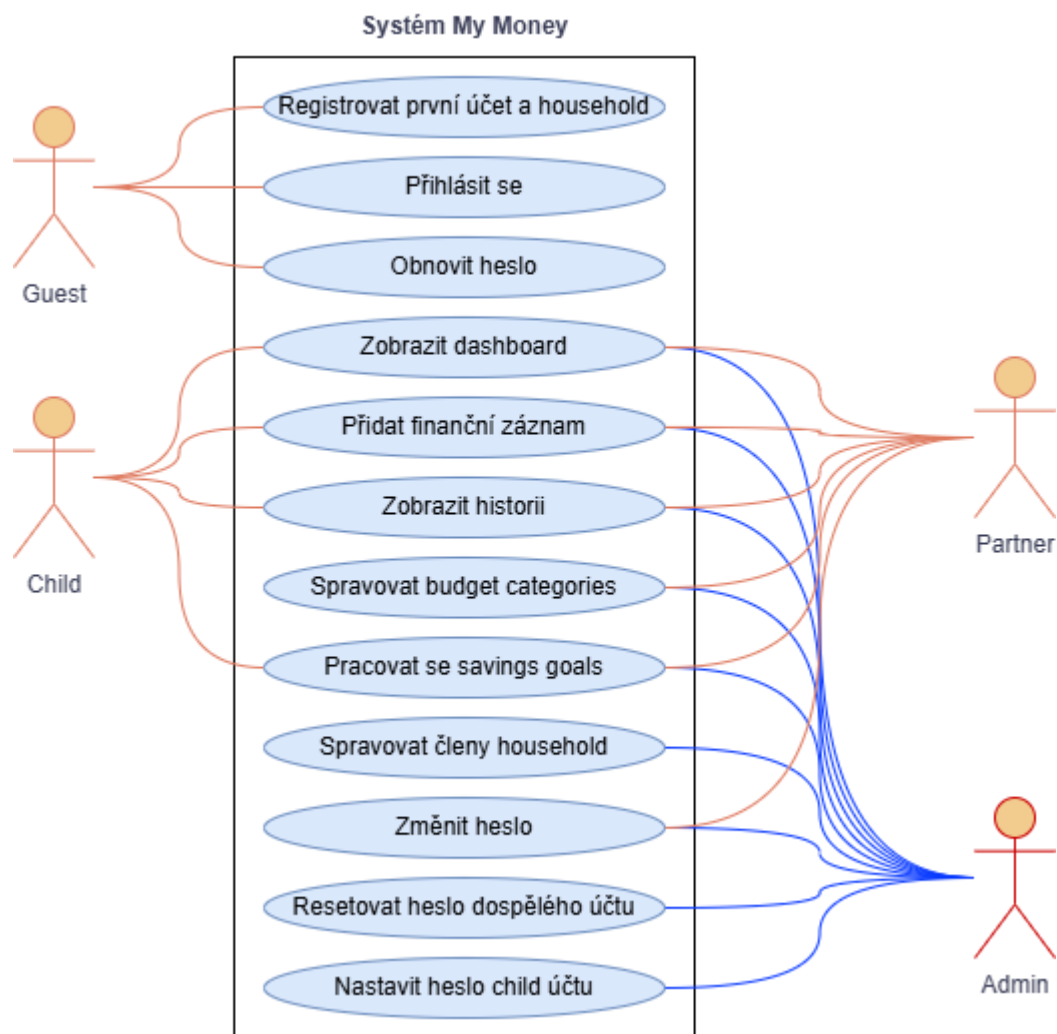
|     |                               |                                                                                                                                              |
|-----|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
|     |                               | nekonzistenci mezi záznamy, zůstatkem a úsporami.                                                                                            |
| B12 | Ochrana lokálně uložených dat | Finanční a uživatelská data budou ukládána lokálně v databázi aplikace a nebudou bezdůvodně přenášena do externích služeb.                   |
| B13 | Řízení uživatelské relace     | Aplikace bude pracovat s aktuální uživatelskou relací a bude oddělovat data a funkce podle přihlášeného uživatele a jeho household kontextu. |

### 3.3 Případy užití

Pro doplnění analýzy systému je vhodné zachytit také hlavní případy užití aplikace My Money. Use case diagram ukazuje základní interakce mezi systémem a jednotlivými typy uživatelů a zároveň vystihuje rozdíly mezi nepřihlášeným uživatelem (Guest), administrátorem, partnerem a child účtem. Tímto způsobem doplňuje funkční požadavky o přehled hlavních činností, které mohou jednotliví aktéři v systému provádět.

V aplikaci vystupují čtyři hlavní typy aktérů. Nepřihlášený uživatel (Guest) může provést registraci prvního účtu a household, přihlášení do systému nebo obnovu hesla. Po úspěšném přihlášení uživatel vstupuje do systému v jedné z rolí Admin, Partner nebo Child. Administrátor má nejširší oprávnění a zajišťuje správu household prostředí. Partner představuje dospělého člena household, který pracuje s osobními i sdílenými financemi podle přidělených oprávnění. Child účet je oproti tomu omezen na vybrané sdílené funkce a nemá přístup ke všem částem systému.

Mezi hlavní případy užití patří zobrazení dashboardu, práce s historií záznamů, přidávání finančních záznamů, správa rozpočtových kategorií, práce s cíli spoření a správa uživatelského účtu. Vedle těchto základních činností diagram zachycuje i případy užití související přímo s household modelem, zejména správu členů household, změnu rolí a oprávnění, reset přístupu dospělých účtů a nastavování hesel child účtům. Rozdíly mezi rolemi se tak neprojevují pouze na úrovni dat, ale i na úrovni samotného používání systému.



**Obrázek 3:** Use case diagram aplikace My Money se znázorněním aktérů Guest, Admin, Partner a Child a hlavních případů užití systému (zdroj: vlastní)

Na úrovni dospělých účtů je důležitá také možnost práce s různými finančními kontexty. Uživatel může podle situace pracovat s personal nebo household částí systému, zatímco child účet zůstává omezen na sdílený prostor. To se promítá například do práce s finančními záznamy, cíli spoření, dashboardem nebo dostupností některých funkcí v historii a správě kategorií.

Use case diagram tak pomáhá lépe vystihnout praktický rozsah navrhovaného řešení a ukazuje, že aplikace My Money není pouze jednoduchým osobním evidenčním nástrojem, ale víceuživatelským systémem s rozdílnými rolemi, oprávněními a scénáři použití.



## 4 NÁVRH

Na základě předchozí analýzy požadavků byl navržen způsob řešení aplikace My Money z hlediska architektury, datového modelu, uživatelského rozhraní a toku dat mezi jednotlivými částmi systému. Cílem této části bylo převést stanovené požadavky do konkrétní podoby řešení, které bude přehledné, funkční a vhodné pro následnou implementaci.

### 4.1 Návrh architektury

Při návrhu aplikace My Money bylo cílem vytvořit řešení, které bude přehledné svou strukturou, rozšiřitelné a vhodné pro realizaci víceuživatelského finančního systému s lokální databází. Z tohoto důvodu byla aplikace navržena jako desktopová aplikace na platformě .NET s využitím WPF, architektonického přístupu MVVM a také servisní a repozitářové vrstvy. [6] [13]

Architektura aplikace je založena na oddělení uživatelského rozhraní, aplikační logiky a vrstvy přístupu k datům. Uživatelské rozhraní je realizováno prostřednictvím View a ViewModel. View odpovídá za zobrazení obrazovek a interakci s uživatelem, zatímco ViewModel spravuje stav obrazovky, připravuje data pro zobrazení a předává uživatelské akce do servisní vrstvy. Díky tomu zůstává vizuální část aplikace oddělena od hlavní logiky systému. [9]

Servisní vrstva obsahuje hlavní obchodní logiku aplikace. Jsou zde realizována pravidla práce s finančními záznamy, kategoriemi rozpočtu, spořicími cíli, členy domácnosti a také mechanismy autorizace, uživatelské relace, analytiky a výpočtu finančního hodnocení. Tato vrstva zároveň odpovídá za kontrolu přístupových práv a omezení operací v závislosti na roli uživatele.

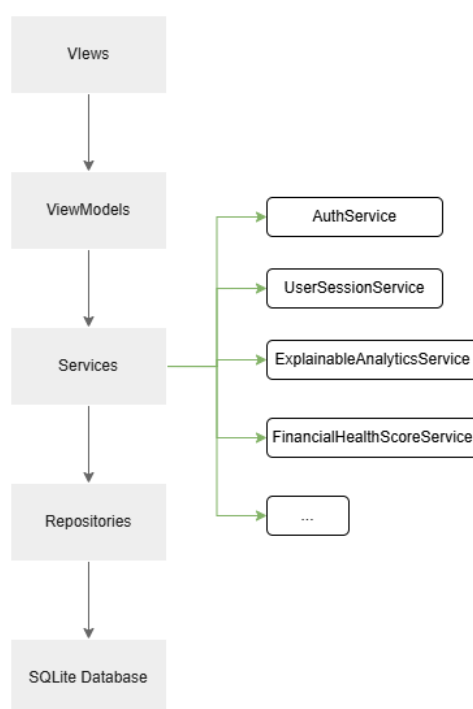
Vrstva přístupu k datům je realizována pomocí repozitářů, které zapouzdřují práci s databází SQLite. Repozitáře provádějí SQL operace a oddělují ukládání dat od vyšších vrstev aplikace. Tento přístup činí projekt přehlednějším, usnadňuje jeho údržbu a umožňuje logicky oddělit práci s jednotlivými entitami systému. [13]

Důležitou součástí architektury je podpora několika finančních kontextů. Aplikace rozlišuje společná data domácnosti, osobní data dospělých uživatelů a sdílenou aktivitu dětských účtů. To ovlivňuje práci s finančními záznamy, kategoriemi rozpočtu, spořicími cíli, nástěnkou a analytikou. Architektura proto musí zohledňovat nejen typ operace, ale také finanční kontext, ve kterém je operace prováděna.

Samostatnou roli hraje modul autentizace a uživatelské relace. Po přihlášení do systému aplikace uchovává údaje o aktuálním uživateli, jeho roli, domácnosti a přístupových právech. Na základě těchto informací se vytváří dostupné rozhraní, seznam povolených akcí a chování jednotlivých částí systému. Díky tomu se aplikace může přizpůsobit rolím Admin, Partner a Child, aniž by bylo nutné vytvářet samostatné verze rozhraní.

Rozšířenou součástí architektury je analytický modul. Zahrnuje nástěnku, grafickou vizualizaci dat, Financial Health Score a pravidlový vysvětlitelný analytický engine. Tento modul analyzuje finanční data, vytváří metriky, grafy a doporučení a činí z aplikace nejen evidenční, ale také analytický systém.

Pro operace ovlivňující finanční stav byla do architektury zahrnuta podpora transakcí. To znamená, že uložení záznamu a změna souvisejícího finančního stavu se provádějí jako jedna atomická operace. Tento přístup snižuje riziko nesouladu dat a zvyšuje spolehlivost aplikace.



**Obrázek 4:** Architektonické schéma aplikace My Money s rozdělením na View, ViewModel, Services, Repositories a SQLite databázi (zdroj: vlastní)

Architektura aplikace My Money je tedy navržena tak, aby podporovala oddělení odpovědností mezi jednotlivými vrstvami, řízení přístupu na základě rolí, práci s několika finančními kontexty a možnost dalšího rozšiřování. Díky tomu je řešení vhodné jak pro praktické použití, tak pro formální popis v rámci bakalářské práce.

## 4.2 Datový model

Datový model aplikace My Money byl navržen s ohledem na to, že systém nepracuje pouze s jedním uživatelem, ale také s domácnostmi, v jejímž rámci může několik členů společně využívat část finančních dat. Z tohoto důvodu musí datová struktura podporovat jak evidenci uživatelů a jejich rolí, tak ukládání finančních informací v různých kontextech.

Základem modelu jsou entity User, Household a HouseholdMember. Entita User ukládá přihlašovací údaje uživatele, včetně jména, e-mailu, hashe hesla a informací o stavu uživatelského účtu. Entita Household představuje rodinnou skupinu nebo společný finanční prostor. Vazba mezi uživatelem a domácností je realizována prostřednictvím entity HouseholdMember, která navíc obsahuje roli člena a jeho přístupová práva. Díky tomu může jedna domácnost zahrnovat několik uživatelů s různými oprávněními.

Pro ukládání finančního stavu se v systému používají dvě samostatné entity: HouseholdFinance a UserFinance. Toto rozdělení umožňuje nezávisle evidovat společné rodinné prostředky a osobní prostředky dospělého uživatele. To je zvláště důležité pro architekturu aplikace, protože část operací musí měnit sdílené finance domácnosti a část pouze osobní finance konkrétního uživatele.

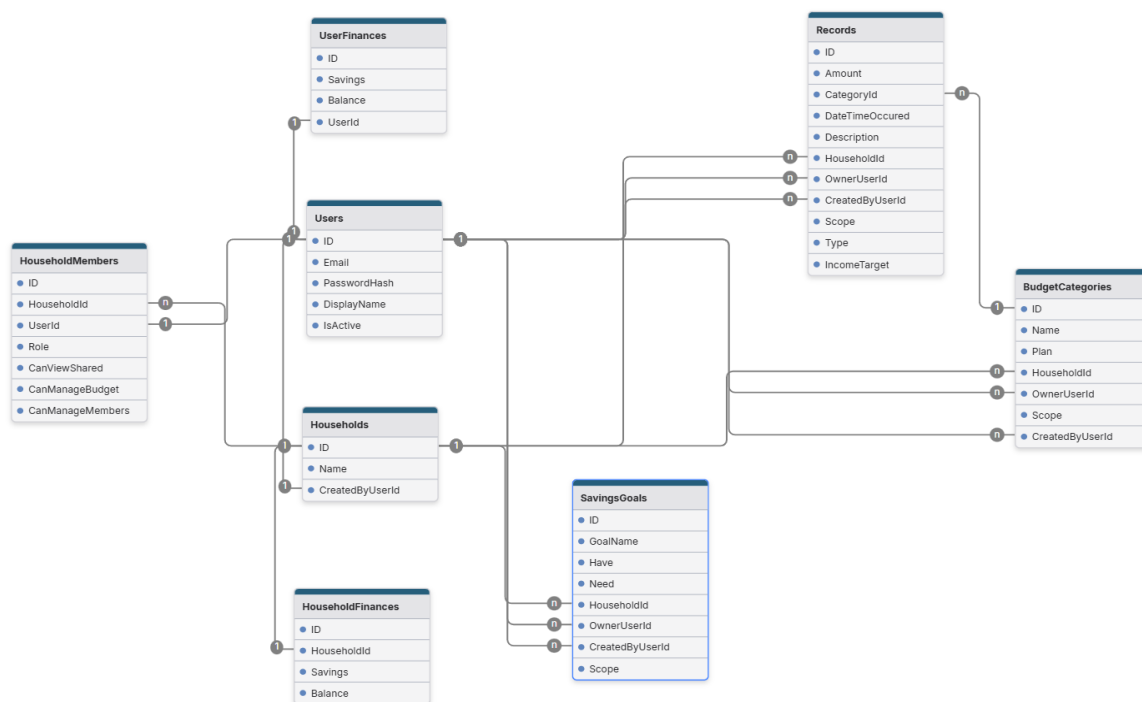
Finanční záznamy jsou ukládány v entitě Record. Každý záznam obsahuje částku, datum, popis, typ operace a vazbu na kategorii. Kromě toho je v záznamu uložena informace o kontextu, ve kterém byl vytvořen, tedy zda se jedná o sdílený nebo osobní záznam. To umožňuje správně rozdělovat záznamy mezi domácnost a osobní část aplikace a zároveň na ně uplatňovat různá pravidla přístupu.

Kategorie rozpočtu jsou reprezentovány entitou BudgetCategory. Kategorie obsahuje název, plánovanou částku a kontext použití. Díky tomu systém podporuje jak společné kategorie domácnosti, tak osobní kategorie pro dospělé uživatele. Toto řešení umožňuje současně vést rodinný rozpočet i individuální výdaje v rámci jedné aplikace.

Pro spoření se používá entita SavingsGoal. Ukládá název cíle, cílovou částku, aktuální pokrok a kontext, ve kterém byl cíl vytvořen. Důležitým prvkem je zde vazba na uživatele, který cíl vytvořil, protože na tom závisí některá omezení, například pravidla pro úpravu cílů u dětských účtů.

Datový model také zahrnuje vazby nezbytné pro analytické chování systému a chování založené na rolích. Finanční záznamy, kategorie a spořicí cíle jsou propojeny buď s domácností, nebo s konkrétním uživatelem, zatímco uživatelské role a práva jsou uloženy

samostatně v členství v domácnosti. To umožňuje aplikační logice přesně určit, kdo objekt vytvořil, komu patří a jaké akce jsou s ním povoleny.



**Obrázek 5:** ER diagram databázového modelu aplikace My Money (zdroj: vlastní)

Při návrhu datového modelu byla důležitým požadavkem také podpora integrity dat. Z tohoto důvodu jsou mezi hlavními entitami definovány vazby, které umožňují kontrolovat příslušnost finančních objektů k domácnosti nebo uživateli a zajišťovat správné fungování servisní vrstvy. Taková datová struktura dobře odpovídá víceuživatelskému charakteru aplikace a umožňuje podporovat jak běžný finanční přehled, tak i analytické a přístupové mechanismy systému.

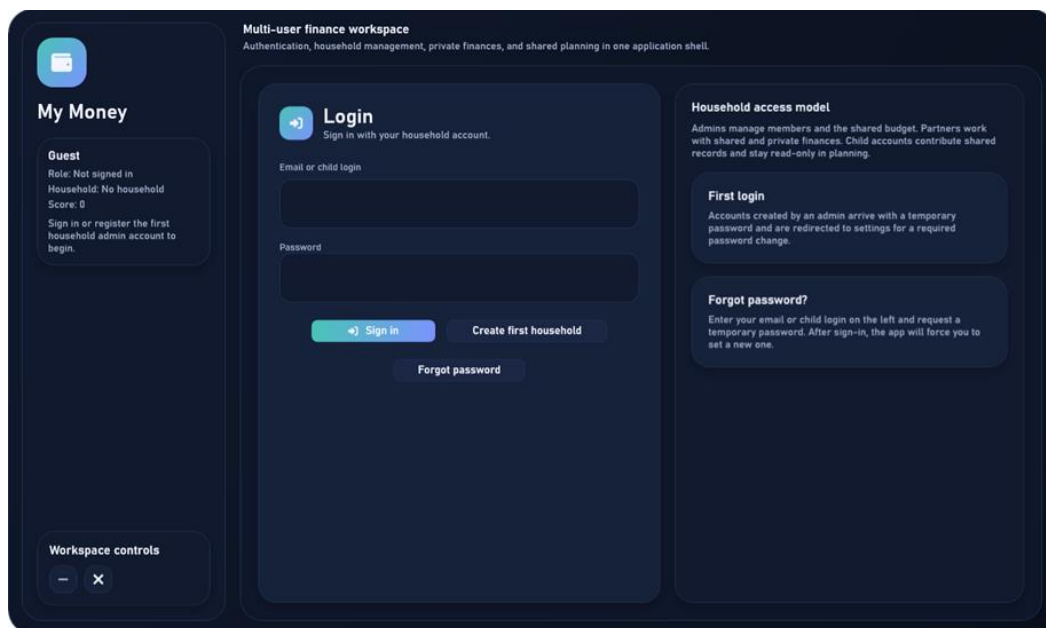
### 4.3 Návrh uživatelského rozhraní

Návrh uživatelského rozhraní aplikace My Money vycházel z požadavku na přehlednou a snadno ovladatelnou desktopovou aplikaci, která bude vhodná pro každodenní práci s osobními i rodinnými financemi. Uživatelské rozhraní proto bylo navrženo tak, aby jednotlivé části systému byly logicky oddělené, aby uživatel rychle pochopil, v jakém kontextu právě pracuje, a aby měl důležité informace dostupné bez zbytečně složité navigace.

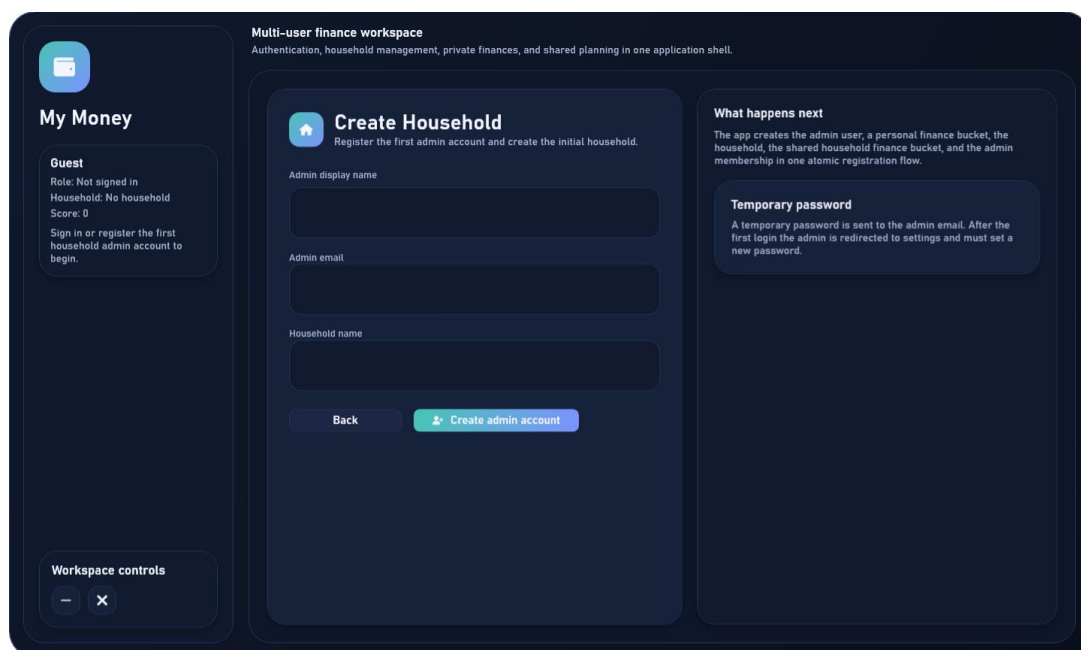
Důležitým principem návrhu bylo rozlišení mezi jednotlivými typy uživatelů a finančními kontexty. Aplikace pracuje s rolemi Admin, Partner a Child, přičemž každá z nich má odlišný rozsah oprávnění. Uživatelské rozhraní se proto nepřizpůsobuje pouze vizuálně, ale také funkčně. Některé akce jsou pro určité role zcela skryté, jiné jsou dostupné pouze v

omezeném rozsahu. Tento přístup zlepšuje přehlednost rozhraní a současně snižuje riziko nesprávné práce se sdílenými daty.

Úvodní část aplikace tvoří přihlašovací a registrační obrazovky. Tyto obrazovky byly navrženy jednoduše a s důrazem na srozumitelnost. Uživatel má na začátku možnost buď vytvořit první household účet, nebo se přihlásit do již existujícího systému. Součástí přihlašovacího rozhraní je také funkce pro obnovení hesla.

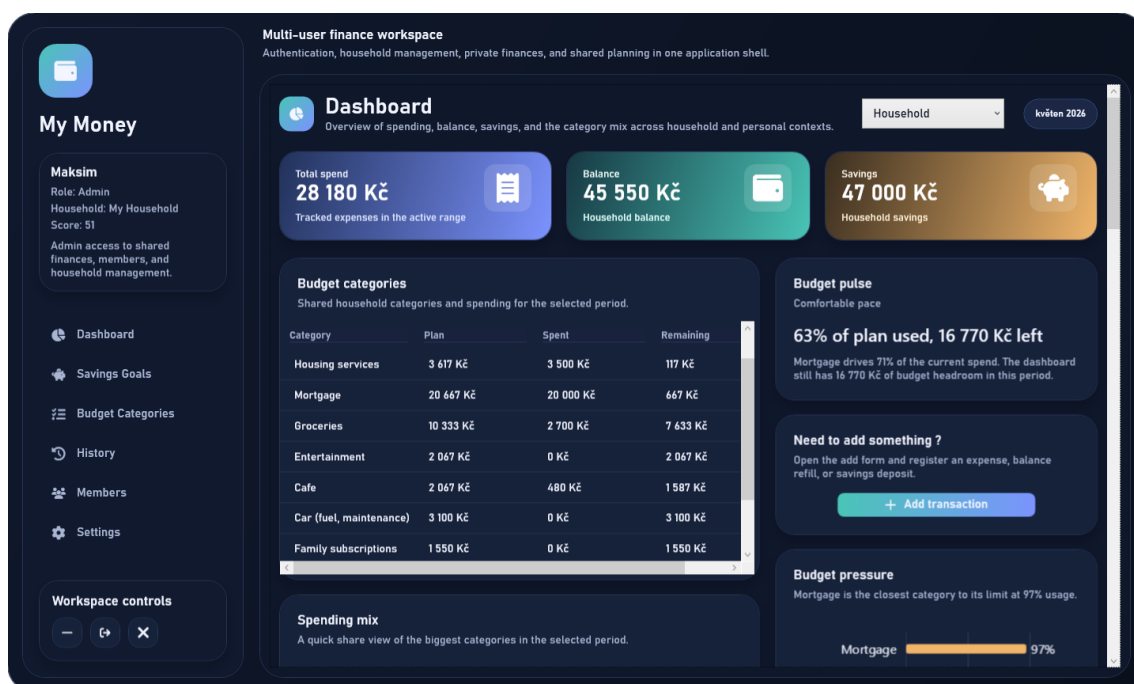


**Obrázek 6:** Přihlašovací obrazovka aplikace My Money (zdroj: vlastní)



**Obrázek 7:** Registrační obrazovka pro vytvoření prvního administrátorského účtu a household (zdroj: vlastní)

Po přihlášení uživatel vstupuje do hlavního aplikačního rozhraní, které tvoří centrální shell s navigací mezi jednotlivými částmi systému. V hlavní části rozhraní se zobrazuje vybraný modul, zatímco navigační oblast umožňuje přechod mezi dashboardem, historií záznamů, plánováním, cíli spoření, správou household členů a nastavením. Součástí hlavního rozhraní jsou také informace o aktuálním uživateli, jeho roli, household a ukazateli Financial Health Score.

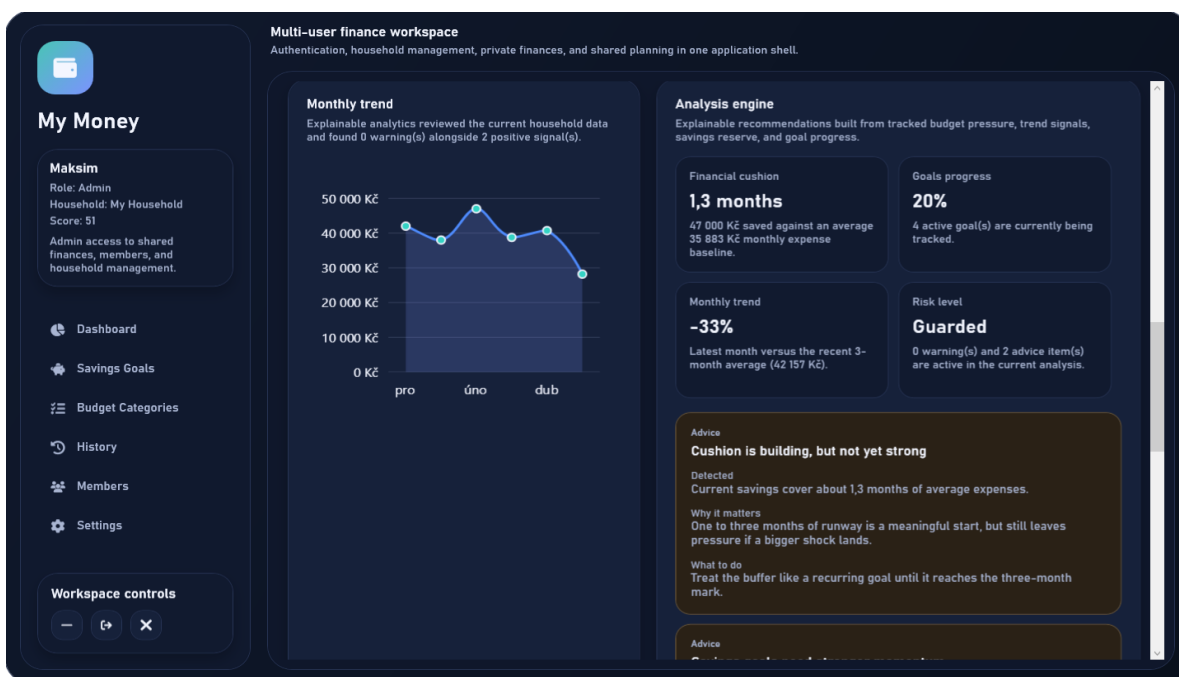


**Obrázek 8:** Hlavní rozhraní aplikace po přihlášení uživatele a dokončení povinné změny hesla (zdroj: vlastní)

Nejvýraznější částí uživatelského rozhraní je dashboard, který plní funkci hlavního analytického panelu aplikace. Dashboard byl navržen tak, aby uživatel na jednom místě viděl základní finanční stav, strukturu výdajů, tlak na rozpočet a vývoj výdajů v čase. Součástí dashboardu jsou grafy Spending mix, Budget pressure a Monthly trend, dále textový blok Budget pulse a sekce Analysis engine, která zobrazuje vysvětlené analytické závěry a doporučení.

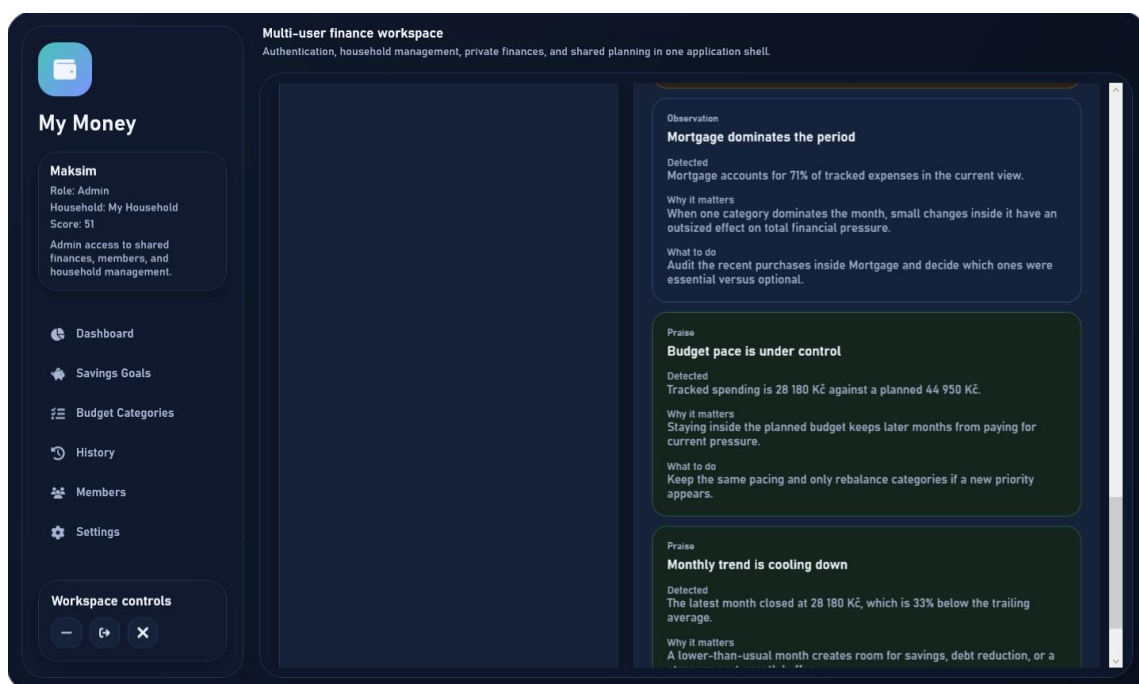


Obrázek 9: Dashboard s grafy Spending mix a Budget pressure (zdroj: vlastní)



Obrázek 10: Dashboard s grafem Monthly trend a sekci Analysis engine s přehledem analytických metrik a doporučení (zdroj: vlastní)





**Obrázek 11:** Pokračování sekce Analysis engine s výstupy typu Observation a Praise (zdroj: vlastní)

Pro práci s běžnými finančními operacemi byly navrženy obrazovky pro zadávání a zobrazení záznamů. Formuláře umožňují zadat částku, datum, kategorii, typ operace a další doplňující údaje. Historie záznamů je zobrazena přehledně a umožňuje filtrovat nebo třídit uložené položky podle zvoleného období. Návrh této části rozhraní byl podřízen tomu, aby uživatel mohl rychle přidat nový záznam a současně se snadno orientovat v již uložených datech.

**Multi-user finance workspace**  
Authentication, household management, private finances, and shared planning in one application shell.

**My Money**

**Maksim**  
Role: Admin  
Household: My Household  
Score: 51  
Admin access to shared finances, members, and household management.

Dashboard  
Savings Goals  
Budget Categories  
History  
Members  
Settings

**Workspace controls**  
— ↻ ✕

**Add entry**  
Create a new household or personal record using the new multi-user finance model.

**Entry setup**  
Pick a scope, choose whether this is an expense or income, and set the target when income should change savings or balance.

**Scope**  
Shared records affect household finances. Personal records stay private to the signed-in user.  
Shared

**Record type**  
Expenses reduce money. Income records add money and can target either balance or savings.  
Expense

**Amount**  
0

**Category**  
Other

**Date**  
10.05.2026

**Description**

Back Save entry

**Obrázek 12:** Formulář pro přidání finančního záznamu (zdroj: vlastní)

**Multi-user finance workspace**  
Authentication, household management, private finances, and shared planning in one application shell.

**My Money**

**Maksim**  
Role: Admin  
Household: My Household  
Score: 51  
Admin access to shared finances, members, and household management.

Dashboard  
Savings Goals  
Budget Categories  
History  
Members  
Settings

**Workspace controls**  
— ↻ ✕

**History**  
Browse shared or private records, switch context, and review the timeline.

**Record timeline**  
Shared household records across all members.

| Date       | Type    | Scope  | Category                | Amount      |
|------------|---------|--------|-------------------------|-------------|
| 10.05.2026 | Expense | Shared | Cafe                    | 480,0 Kč    |
| 07.05.2026 | Expense | Shared | Groceries               | 2 700,0 Kč  |
| 06.05.2026 | Expense | Shared | Internet                | 1 500,0 Kč  |
| 05.05.2026 | Expense | Shared | Housing services        | 3 500,0 Kč  |
| 03.05.2026 | Expense | Shared | Mortgage                | 20 000,0 Kč |
| 01.05.2026 | Income  | Shared | Other                   | 52 000,0 Kč |
| 29.04.2026 | Income  | Shared | Other                   | 1 200,0 Kč  |
| 25.04.2026 | Expense | Shared | Other                   | 1 300,0 Kč  |
| 23.04.2026 | Expense | Shared | Groceries               | 3 400,0 Kč  |
| 20.04.2026 | Expense | Shared | Entertainment           | 1 700,0 Kč  |
| 17.04.2026 | Expense | Shared | Car (fuel, maintenance) | 1 800,0 Kč  |
| 15.04.2026 | Expense | Shared | Family subscriptions    | 1 500,0 Kč  |
| 12.04.2026 | Expense | Shared | Groceries               | 2 900,0 Kč  |

**Context**  
Choose which record stream should be displayed in the timeline.  
Household

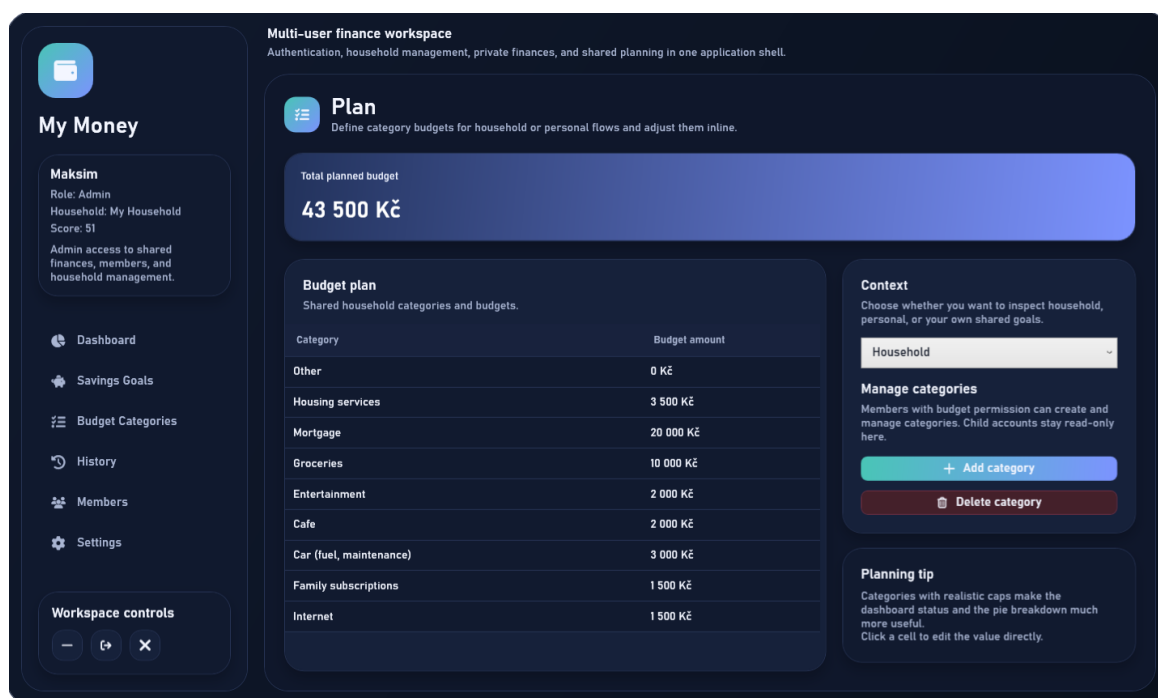
**Sort mode**  
Switch between a date-first and amount-first view.  
Date Amount

**Delete selection**  
Adults can remove entries. Child accounts stay read-only in history.  
Delete record

**Tip**  
When you sort by amount, the most expensive records rise to the top

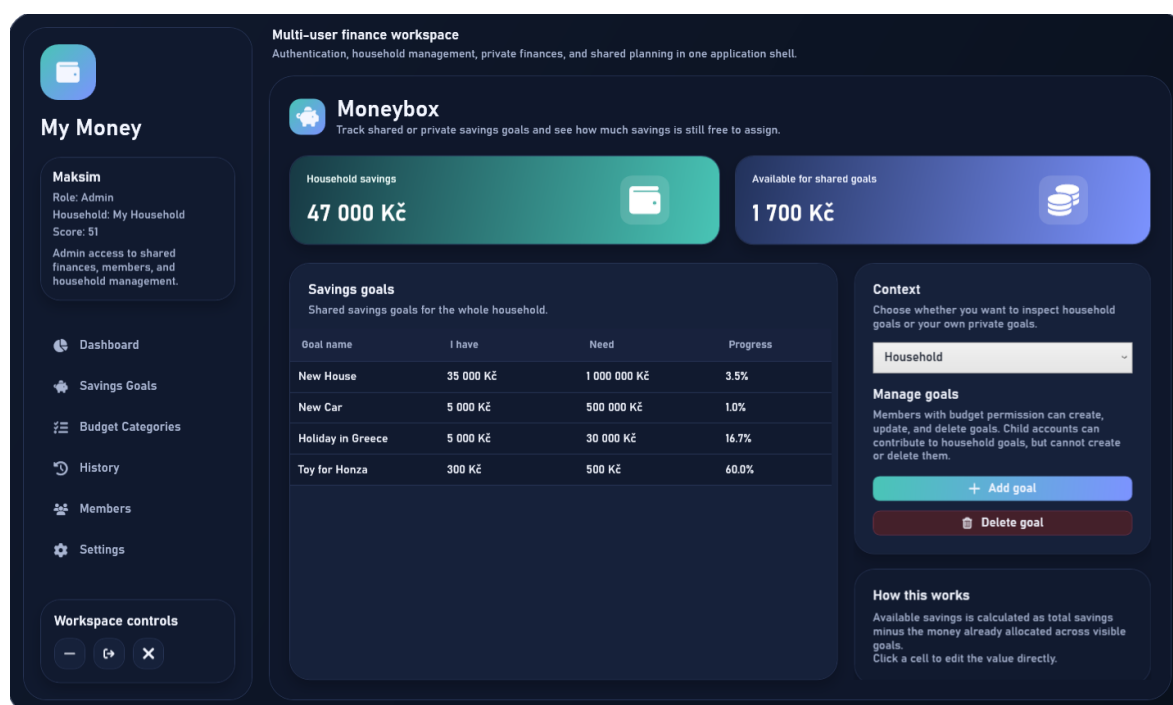
**Obrázek 13:** Obrazovka historie finančních záznamů (zdroj: vlastní)

Samostatná část rozhraní je věnována správě rozpočtových kategorií a plánování. Zde může uživatel pracovat s kategoriemi, nastavovat plánované částky a sledovat jejich vztah ke skutečným výdajům. Vizuální podoba této části byla navržena tak, aby bylo co nejlépe patrné, které kategorie jsou pod kontrolou a které se blíží překročení rozpočtu.



**Obrázek 14:** Obrazovka pro správu budget categories a plánovaných částek (zdroj: vlastní)

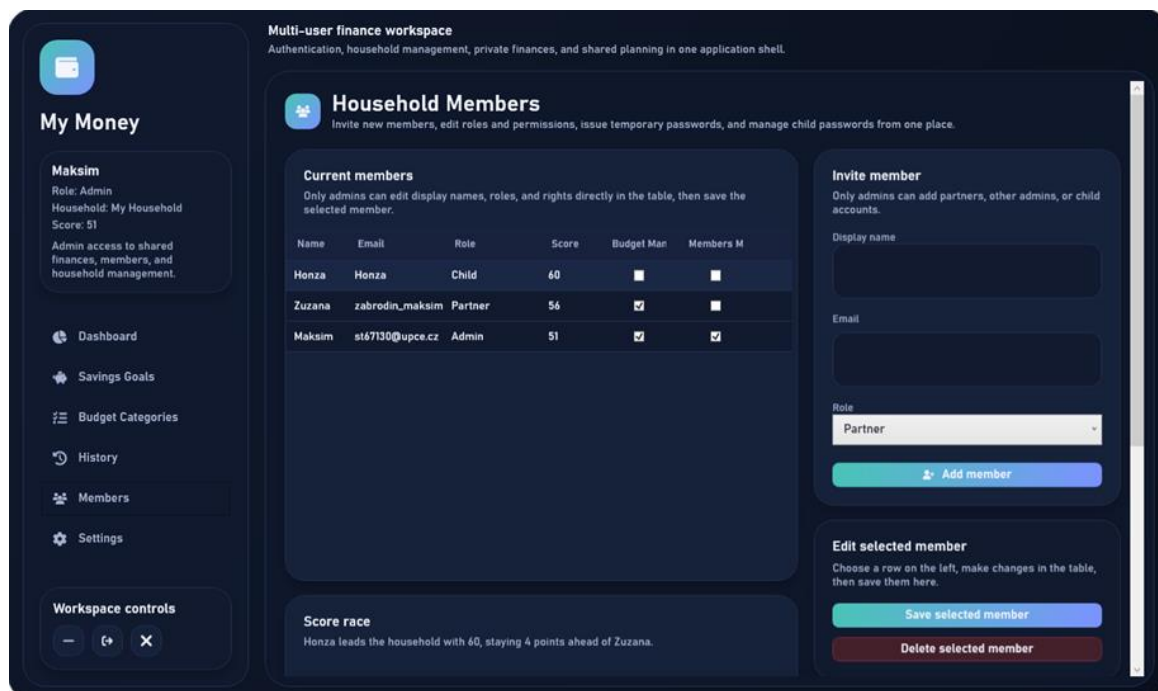
Dalším důležitým modulem je část zaměřená na Savings goals. Uživatelské rozhraní této části umožňuje vytvářet cíle, zobrazovat jejich cílovou částku, aktuální stav a průběžný pokrok. V návrhu bylo důležité zachovat jednoduchost a současně zřetelně oddělit operace, které může provádět běžný dospělý uživatel, od operací dostupných pro child účet.



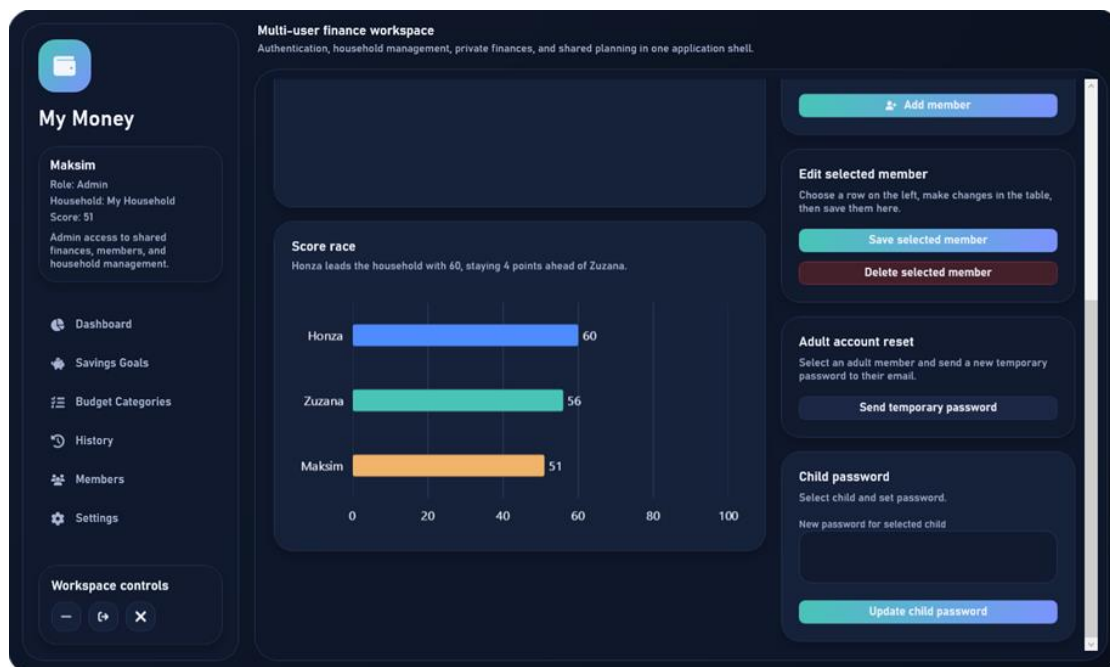
**Obrázek 15:** Obrazovka savings goals s přehledem cílů a jejich plnění (zdroj: vlastní)

Velký význam má také obrazovka pro správu členů household. Tato část rozhraní byla navržena především pro administrátora a obsahuje tabulku členů, jejich role, oprávnění a

hodnotu Financial Health Score. Součástí obrazovky je také grafické porovnání členů household formou ranking nebo racing chart. Tato část rozhraní spojuje správu přístupových práv s analytickým pohledem na finanční chování jednotlivých členů rodiny.



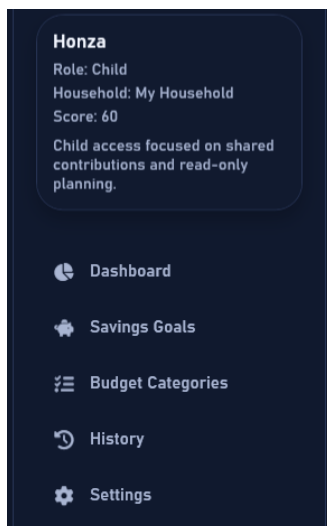
**Obrázek 16:** Obrazovka Household Members s tabulkou členů, rolemi a oprávněními (zdroj: vlastní)



**Obrázek 17:** Grafické porovnání členů household podle Financial Health Score v části Household Members (zdroj: vlastní)

Z hlediska použitelnosti bylo důležité, aby aplikace vizuálně odlišovala akce, které jsou pro daného uživatele dostupné, od těch, které dostupné nejsou. To je patrné zejména u child účtů,

kde jsou některé funkce skryté již na úrovni rozhraní. Typicky se jedná o omezení správy household members, mazání historie nebo vytváření a mazání některých budget categories. Tento způsob návrhu rozhraní zlepšuje uživatelský komfort a současně podporuje bezpečné používání systému.



**Obrázek 18:** Příklad postranní navigace pro child účet s omezenou sadou dostupných funkcí (zdroj: vlastní)

Celkově bylo uživatelské rozhraní aplikace My Money navrženo s důrazem na přehlednost, logické členění obrazovek a propojení běžné evidence financí s analytickými a household funkcemi. Výsledné rozhraní podporuje každodenní práci se systémem, respektuje role jednotlivých uživatelů a současně umožňuje názornou prezentaci finančních dat pomocí moderních vizuálních prvků.

#### 4.4 Návrh navigace a toku dat

Při návrhu aplikace My Money bylo důležité řešit nejen vzhled jednotlivých obrazovek, ale také způsob pohybu uživatele mezi nimi a tok dat mezi vrstvami systému. Navigace i tok dat proto byly navrženy tak, aby aplikace působila přirozeně, byla snadno ovladatelná a současně odpovídala vícevrstvé architektuře.

Navigace v aplikaci vychází z toho, zda je uživatel přihlášený a jakou roli v household zastává. Pokud v systému ještě neexistuje žádný uživatel, aplikace nabídne registrační obrazovku pro vytvoření prvního household účtu. V opačném případě se po spuštění zobrazí přihlašovací obrazovka. Teprve po úspěšném přihlášení získá uživatel přístup do hlavního rozhraní aplikace, a tím i k finančním a household funkcím systému.

Po přihlášení se uživatel pohybuje v hlavním aplikačním shellu, který obsahuje centrální navigaci mezi jednotlivými moduly. Z této úrovně lze přecházet na dashboard, historii

záznamů, plánování rozpočtu, cíle spoření, správu členů household a nastavení. Důležitou vlastností navigace je její přizpůsobení konkrétnímu typu uživatele. Administrátor má například přístup ke správě členů household a k úpravám oprávnění, zatímco child účet má část funkcí omezenou nebo skrytou. Navigace proto neplní jen roli technického přechodu mezi obrazovkami, ale je i součástí logiky přístupových práv.

Vedle samotné navigace byl při návrhu důležitý také tok dat mezi jednotlivými vrstvami aplikace. Uživatelská akce vzniká na úrovni View, například kliknutím na tlačítko, změnou filtru nebo vyplněním formuláře. Tato akce je zpracována ve ViewModelu, který připraví požadavek pro příslušnou službu. Servisní vrstva následně provede potřebnou logiku, ověří pravidla přístupu a podle potřeby zavolá repozitář, který načte nebo uloží data do databáze. Výsledek se poté vrací zpět přes servisní vrstvu do ViewModelu a následně do uživatelského rozhraní. Takový tok dat odpovídá architektuře MVVM a pomáhá udržet projekt přehledný.

Důležitou roli v tomto procesu hraje uživatelská relace. Po přihlášení se ukládají informace o aktuálním uživateli, jeho household členství, roli a oprávněních. Tyto údaje následně ovlivňují navigaci i samotné načítání dat. Dashboard, historie nebo savings goals se proto nenačítají obecně, ale vždy v určitém kontextu podle toho, zda uživatel pracuje s household daty, personal daty nebo child shared aktivitou. Stejná obrazovka tak může podle kontextu zobrazovat jiná data a nabízet jiný rozsah operací.

Samostatnou část toku dat tvoří analytický modul. Dashboard nezobrazuje pouze základní přehledy, ale také výstupy z Financial Health Score a explainable analytics engine. Tyto části pracují nad daty získanými z více služeb současně, například ze záznamů, kategorií, cílů spoření a finančního stavu uživatele nebo household. Výsledkem jsou souhrnné ukazatele, grafy a doporučení, které se následně zobrazují v přehledné podobě na jedné obrazovce.

Při návrhu navigace a toku dat bylo cílem, aby se uživatel v aplikaci pohyboval bez zbytečných překážek, zatímco vnitřní logika systému zůstane oddělená a dobře organizovaná. Aplikace je tak srozumitelná z pohledu používání a současně stabilní z pohledu implementace.

## 5 IMPLEMENTACE

Po návrhu architektury, datového modelu a uživatelského rozhraní následovala samotná implementace aplikace My Money. Cílem této fáze bylo převést navržené řešení do funkční desktopové aplikace, která bude odpovídat stanoveným požadavkům a současně zachová přehlednou vnitřní strukturu projektu.

Implementace byla realizována na platformě .NET v jazyce C# s využitím technologie WPF. Při tvorbě byl použit architektonický přístup MVVM, doplněný o servisní a repozitářovou vrstvu. Toto řešení umožnilo oddělit uživatelské rozhraní od aplikační logiky a práce s databází. Takové uspořádání se ukázalo jako důležité zejména s ohledem na rostoucí rozsah projektu. Aplikace se totiž v průběhu práce rozšířila z jednoduchého osobního finančního nástroje o víceuživatelský household model, správu rolí, autentizaci, analytický modul a základní testování.

Při implementaci byl kladen důraz nejen na samotnou funkčnost, ale také na to, aby jednotlivé části systému byly logicky rozdělené a dobře navazovaly na návrhovou část. Zvláštní pozornost byla věnována správě uživatelské relace, rozdělení dat podle kontextu, omezení přístupu podle rolí a konzistentní práci s finančními operacemi. Důležitou součástí implementace se stal také analytický dashboard a modul vysvětlitelné analýzy, který zpracovává finanční data do podoby přehledných metrik, grafů a doporučení.

V následujících podkapitolách je proto popsána struktura projektu, hlavní implementační principy a vybrané části řešení, které mají z hlediska výsledné aplikace největší význam.

### 5.1 Použité nástroje a technologie vývoje

Při tvorbě aplikace My Money byly využity jak hlavní programové technologie určující architekturu a realizaci systému, tak i podpůrné nástroje usnadňující správu projektu, ladění a verzování zdrojového kódu. Volba těchto prostředků vycházela z charakteru projektu, který představuje desktopovou aplikaci na platformě .NET s lokálním ukládáním dat a grafickým uživatelským rozhraním.

Použité nástroje musely především zajistit pohodlnou práci s jazykem C# a prostředím WPF, efektivní organizaci projektu, podporu ladění a současně i správu verzí zdrojového kódu. Důležitá byla také jejich vzájemná kompatibilita, aby práce probíhala v jednotném a stabilním technologickém prostředí.

Jako hlavní vývojové prostředí bylo při realizaci aplikace použito Visual Studio. Toto IDE patří mezi nejrozšířenější nástroje pro práci na platformě .NET a poskytuje podporu pro

jazyk C#, tvorbu desktopových aplikací ve WPF, správu projektů, závislostí, ladění i analýzu kódu. Pro bakalářský projekt zaměřený na návrh a implementaci desktopové aplikace představovalo přirozenou a praktickou volbu. [26]

Významnou předností prostředí Visual Studio je jeho přímá vazba na technologie použité v projektu. Umožňuje pohodlnou práci se soubory XAML, přehlednou organizaci solution i jednotlivých tříd a poskytuje nástroje pro ladění aplikační logiky i uživatelského rozhraní. V jednom prostředí tak bylo možné soustředit prezentační vrstvu, aplikační logiku i část přístupu k datům.

Důležitou roli sehrála také správa balíčků a závislostí. Protože aplikace využívá vedle standardních součástí .NET i další knihovny, bylo nutné mít k dispozici nástroj pro jejich snadné přidávání a správu. Visual Studio v tomto směru poskytuje přímou podporu pro práci s balíčky NuGet, a tím přispívá k přehlednější a efektivnější správě projektu. [27]

Pro správu zdrojového kódu byl použit systém Git a jako vzdálené úložiště sloužila platforma GitHub. V současné softwarové praxi představují tyto nástroje běžnou součást práce, protože umožňují evidovat změny v projektu, uchovávat jejich historii a bezpečně spravovat jednotlivé verze zdrojového kódu. [28]

Použití systému Git mělo význam zejména z hlediska systematické práce na projektu. V průběhu realizace aplikace se struktura programu postupně měnila, přibývaly nové funkce, upravovala se architektura a probíhaly opravy chyb. Systém správy verzí umožnil tyto změny zaznamenávat a v případě potřeby se vracet k předchozím stavům projektu. [28]

Platforma GitHub doplnila systém Git o vzdálené úložiště repozitáře a usnadnila zálohování i organizaci projektu. V kontextu bakalářské práce mělo její použití také dokumentační význam, protože umožnilo uchovat projekt v ucelené podobě a zachytit jednotlivé etapy jeho realizace. [29]

Systém Git a platformu GitHub tak lze považovat za důležitou součást technologického zázemí projektu. Jejich využití přispělo k lepší organizaci práce, bezpečnější správě zdrojového kódu a přehlednějšímu zachycení postupu realizace aplikace.

## **5.2 Struktura projektu**

Projekt aplikace My Money je rozdělen do několika logických částí, které odpovídají zvolené architektuře a usnadňují orientaci ve zdrojovém kódu. Vzhledem k tomu, že aplikace zahrnuje uživatelské účty, household model, finanční logiku, analytiku i práci s databází, bylo nutné zachovat ve struktuře projektu jasné rozdělení odpovědností.



Základní část tvoří složky Views a ViewModel. Ve složce Views jsou umístěny jednotlivé obrazovky aplikace, například přihlašování, dashboard, historie záznamů, správa household members, savings goals nebo nastavení. Složka ViewModel obsahuje odpovídající ViewModel, které spravují stav obrazovek, reagují na akce uživatele a připravují data pro zobrazení. Právě zde se promítá použití vzoru MVVM, protože prezentační logika je oddělena od samotného vzhledu obrazovek.

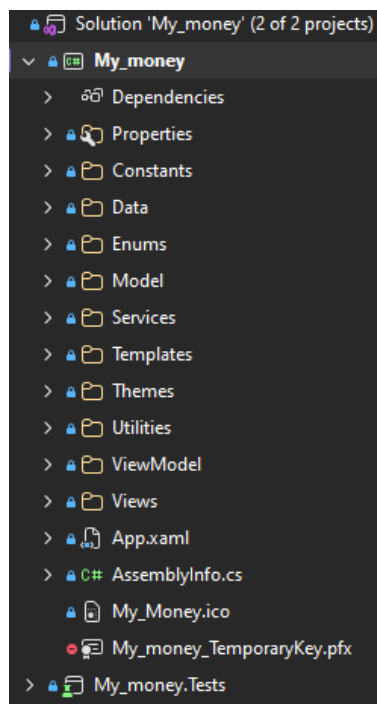
Datová a doménová část aplikace je soustředěna ve složce Model. Nacházejí se zde entity reprezentující hlavní objekty systému, například uživatel, household, člen household, finanční záznam, rozpočtová kategorie nebo cíl spoření. Tyto třídy tvoří základní datovou strukturu aplikace a slouží jako společný model pro další vrstvy systému.

Významnou roli mají složky Services a Repositories. Repozitáře zajišťují komunikaci s databází SQLite a obsahují operace pro načítání, ukládání, aktualizaci a mazání dat. Servisní vrstva nad nimi realizuje aplikační logiku, například práci s rolemi, správu household, finanční záznamy, rozpočtové kategorie, cíle spoření, uživatelskou relaci, autentizaci nebo analytické vyhodnocení finanční situace.

Součástí projektu jsou také pomocné části, například Enums, Constants a utility třídy. Tyto části obsahují výčetové typy, sdílené konstanty a pomocné mechanismy využívané napříč aplikací.

Důležitým místem celé aplikace je startovní konfigurace projektu, kde dochází k registraci služeb, repozitářů a ViewModel v rámci Dependency Injection. Současně se zde inicializuje databáze a nastavuje výchozí tok aplikace podle toho, zda v systému existuje alespoň jeden uživatel.

Vedle hlavního projektu byl vytvořen také samostatný testovací projekt My\_money.Tests, který slouží k ověřování vybraných částí aplikační logiky pomocí unit testů, zejména v oblasti analytických pravidel.



**Obrázek 19:** Struktura solution aplikace My Money v prostředí Visual Studio (zdroj: vlastní)

### 5.3 Autentizace a uživatelská relace

Jednou z důležitých částí implementace aplikace My Money je autentizace uživatelů a práce s uživatelskou relací. Tato část systému je nezbytná proto, že aplikace nepodporuje pouze jednoho lokálního uživatele, ale více členů household s různými rolmi a oprávněními. Bylo proto nutné zajistit, aby se po spuštění systému správně určilo, kdo je přihlášen, do jakého household patří a jaké funkce může používat.

Přihlášení uživatele je založeno na kombinaci e-mailu a hesla. Při pokusu o přihlášení aplikace nejprve ověří, zda účet s daným e-mailem existuje, a následně porovná zadané heslo s uloženou hashovanou hodnotou. Pokud je ověření úspěšné, systém načte také odpovídající household členství a vytvoří aktivní uživatelskou relaci. Tím může aplikace od začátku pracovat nejen s identitou uživatele, ale i s jeho rolí a oprávněními v rámci household.

Při prvním použití aplikace je k dispozici registrační scénář pro vytvoření prvního účtu a současné založení household. Tento uživatel získává roli administrátora a stává se základním správcem systému. V dalších krocích může právě administrátor přidávat další členy household, a to jako dospělé nebo child účty.

Součástí autentizační logiky je také obnova hesla. U dospělých účtů je tento proces řešen zasláním dočasného hesla na e-mail uživatele. Po přihlášení s tímto heslem je uživatel vyzván k nastavení nového hesla. Pro child účty je zvolen odlišný přístup, protože tyto účty

spravuje administrátor household. Heslo dítěte proto může nastavovat nebo měnit pouze administrátor přímo z rozhraní pro správu household members.

A new account has been created for you.

Temporary password:  
ED64Nj6NEg

Please log in and change your password as soon as possible.

For security reasons, do not share this password with anyone.

If you did not expect this account, please ignore this email or contact support.

Support Team myMoney app : st67130@upce.cz

**Obrázek 20:** Ukázka e-mailové zprávy s dočasným heslem při vytvoření nového uživatelského účtu (zdroj: vlastní)

You have requested to reset your password.

Temporary password:  
%@HBn5bSHe

Please log in using this password and set a new one immediately.

If you did not request a password reset, you can safely ignore this email.

Support Team myMoney app : st67130@upce.cz

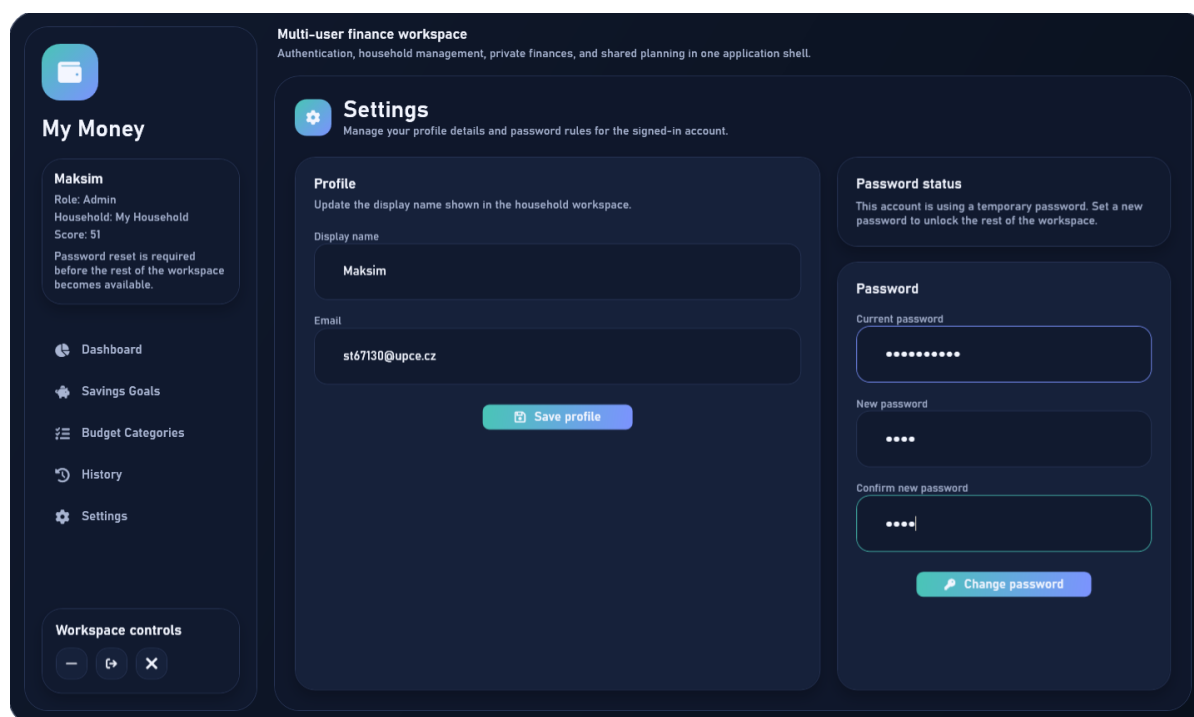
**Obrázek 21:** Ukázka e-mailové zprávy s dočasným heslem při obnovení přístupu do aplikace (zdroj: vlastní)

Po úspěšném přihlášení aplikace uchovává informace o aktuálním uživateli v rámci uživatelské relace. Relace obsahuje především údaje o identitě uživatele, jeho household členství, roli a dalších oprávněních. Tyto informace následně využívají další části systému, například navigace, dashboard, správa household members nebo omezení funkcí pro child účty. Relace tak nepředstavuje pouze pomocný technický prvek, ale základní mechanismus, na kterém je postaveno chování aplikace podle rolí.

Implementace relace zároveň zajišťuje, že se jednotlivé moduly aplikace vždy načítají ve správném kontextu. Systém rozlišuje, zda má uživatel pracovat s personal daty, household daty nebo s omezenou shared aktivitou child účtu. Bez této vazby by nebylo možné správně filtrovat data, řídit oprávnění ani zobrazovat odpovídající obsah jednotlivým typům uživatelů.

Důležitým prvkem je také odhlášení, při kterém dojde k ukončení aktivní relace a vyčištění informací o aktuálním uživateli. Tím je zajištěno, že při dalším použití aplikace nebude přístup k datům předchozího uživatele zachován. Významnou součástí zabezpečení je i

povinná změna hesla po přihlášení s dočasným heslem, během níž zůstávají ostatní navigační funkce neaktivní.



**Obrázek 22:** Obrazovka povinné změny hesla po přihlášení s dočasným heslem; navigační funkce jsou do dokončení změny hesla neaktivní (zdroj: vlastní)

## 5.4 Role a přístupová práva

Protože je aplikace My Money navržena jako víceuživatelský household systém, bylo nutné v implementaci řešit také role a přístupová práva. Tato část určuje, jaké funkce jsou dostupné jednotlivým uživatelům, s jakými daty mohou pracovat a které operace smějí provádět. Nestačilo tedy pouze odlišit běžného uživatele od administrátora, ale bylo potřeba navrhnout jemnější model oprávnění odpovídající rodinnému prostředí.

V systému jsou implementovány tři základní role: Admin, Partner a Child. Administrátor představuje hlavního správce household a má nejširší oprávnění. Partner je dospělý člen household, který může pracovat s osobními i sdílenými daty v rozsahu přidělených práv. Child účet představuje omezený typ uživatele, který pracuje pouze se sdílenou částí systému a nemá přístup k některým organizačním nebo citlivějším funkcím.

Vedle samotné role využívá aplikace také doplňková oprávnění, zejména CanManageBudget a CanManageMembers. Chování systému tak není určováno jen názvem role, ale i konkrétní sadou práv, která jsou uživateli přidělena. Tento přístup umožňuje přesněji řídit, co může který člen household spravovat.

Logika oprávnění se projevuje na více úrovních. V uživatelském rozhraní jsou některé akce nebo celé části aplikace skryté, pokud k nim uživatel nemá přístup. Typickým příkladem je správa členů household, úpravy rozpočtových kategorií nebo mazání historie. Zároveň však nejsou omezení řešena pouze vizuálně. Stejná pravidla jsou kontrolována také v servisní vrstvě, která rozhoduje o tom, zda bude konkrétní operace skutečně provedena. Přístupová práva jsou tak vynucována v jádru systému, nejen na úrovni rozhraní.

Specifickou část implementace tvoří pravidla pro child účty. Tyto účty nemají přístup k personal finance kontextu a pracují pouze se sdílenou částí household. Nemohou spravovat členy household, mazat záznamy historie ani plně spravovat budget categories. V oblasti savings goals mají omezená, ale smysluplná oprávnění: mohou vytvářet vlastní shared cíle, přispívat na rodinné cíle a upravovat pouze ty cíle, které samy založily.

Administrátor má naopak nejširší pravomoci. Může přidávat a odstraňovat členy household, měnit jejich role, upravovat oprávnění a spravovat přístupové údaje child účtů. Partner představuje střední úroveň oprávnění. Může pracovat se svými personal daty a podle přidělených práv také s vybranými shared funkcemi. Tím lze v household nastavit přirozené rozdělení odpovědností.

Z implementačního hlediska je důležité, že role a oprávnění přímo ovlivňují navigaci, načítání dat, dostupnost tlačítek, možnosti editace i analytické zobrazení. Aplikace se tak přizpůsobuje postavení uživatele v household a nepracuje se všemi členy stejným způsobem.

## 5.5 Repositories a Services

Důležitou součástí implementace aplikace My Money tvoří repozitáře a servisní vrstva. Tyto dvě části zajišťují, že je projekt rozdělen přehledně a že logika aplikace není soustředěna přímo v uživatelském rozhraní. Vzhledem k rozsahu výsledného řešení by nebylo vhodné umisťovat práci s databází nebo pravidla přístupových práv do ViewModel, protože by to vedlo k nepřehlednému a hůře udržitelnému kódu.

Repozitáře slouží jako vrstva přímé komunikace s databází SQLite. Každý repozitář je zaměřen na určitou skupinu entit a zajišťuje načítání, ukládání, aktualizaci a mazání dat. Jejich úkolem je zapouzdřit SQL dotazy a oddělit databázové operace od vyšší aplikační logiky.

Nad repozitáři je postavena servisní vrstva, která zpracovává vlastní logiku systému. Servisy rozhodují o tom, jak budou data použita, v jakém pořadí se provedou jednotlivé operace a zda má uživatel právo konkrétní akci vykonat. Právě zde se promítá většina pravidel typických pro aplikaci My Money, například registrace uživatele a vytvoření household,

správa relace, omezení child účtů, práce s personal a shared daty, finanční operace, správa rolí nebo logika analytického modulu.

Výhodou tohoto rozdělení je, že repozitáře řeší technickou stránku práce s daty, zatímco servisy se soustředí na význam operace z pohledu aplikace. U finančního záznamu například nestačí pouze uložit novou položku do databáze. Servis zároveň řeší, v jakém kontextu se záznam zapisuje, jaký má typ, zda smí být vytvořen daným uživatelem a zda se při jeho uložení musí změnit také zůstatek nebo úspory.

Významnou roli zde hraje také Dependency Injection. Jednotlivé služby, repozitáře a ViewModel nejsou vytvářeny přímo uvnitř jiných tříd, ale jsou registrovány při spuštění systému a následně předávány tam, kde jsou potřeba. Tento přístup omezuje pevnou provázanost mezi komponentami a usnadňuje další rozšiřování i testování aplikace.

Servisní vrstva má velký význam také kvůli přístupovým právům. Omezení pro role Admin, Partner a Child nejsou navázána pouze na rozhraní, ale jsou kontrolována i při samotném provedení operací. Servis tak rozhoduje například o tom, zda child účet může změnit konkrétní savings goal, zda může smazat záznam historie nebo zda má uživatel přístup k určité části shared dat.

Zvláštní skupinu tvoří servisy, které nezajišťují jen běžnou správu dat, ale i vyšší funkce systému. Patří sem například služby pro autentizaci, reset hesla, správu uživatelské relace, výpočet Financial Health Score nebo explainable analytics engine. Servisní vrstva tak nepředstavuje pouze pomocnou část aplikace, ale skutečné centrum její logiky.

Z implementačního hlediska se tento přístup ukázal jako velmi vhodný. ViewModel zůstaly přehlednější, databázová logika byla soustředěna na jednom místě a složitější pravidla aplikace bylo možné lépe organizovat i dále rozšiřovat.

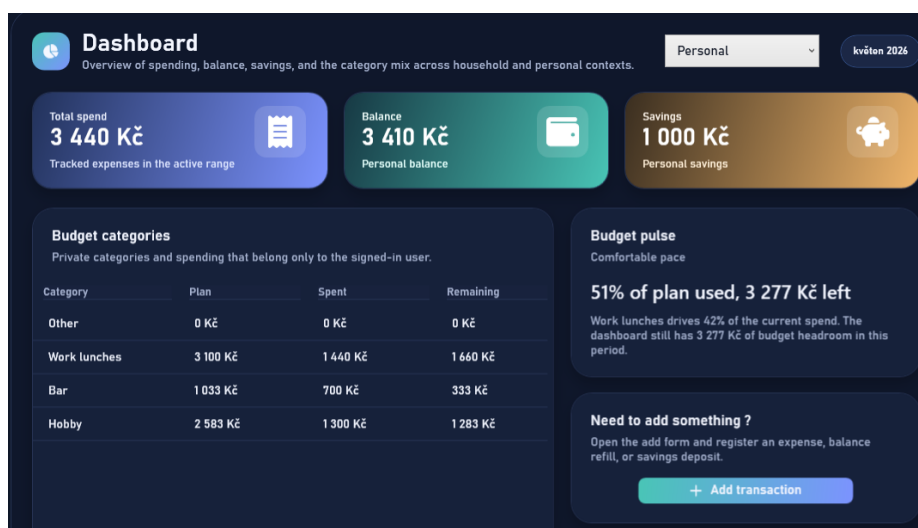
## **5.6 Dashboard, analytika a Financial Health Score**

Jednou z nejvýraznějších částí celé implementace je dashboard a s ním spojený analytický modul. Původním cílem dashboardu bylo zobrazit základní finanční přehled, tedy výdaje, zůstatek a úspory. V průběhu vývoje se však tato část aplikace výrazně rozšířila a dnes plní roli hlavního analytického centra systému. Uživatel zde nevidí pouze jednotlivé hodnoty, ale také jejich vztahy, vývoj v čase a stručnou interpretaci aktuální finanční situace.

Dashboard pracuje s více datovými kontexty. Dospělý uživatel může podle situace zobrazit household data nebo vlastní personal data, zatímco child účet pracuje pouze v omezeném shared kontextu. To znamená, že stejná obrazovka může podle aktivního uživatele a

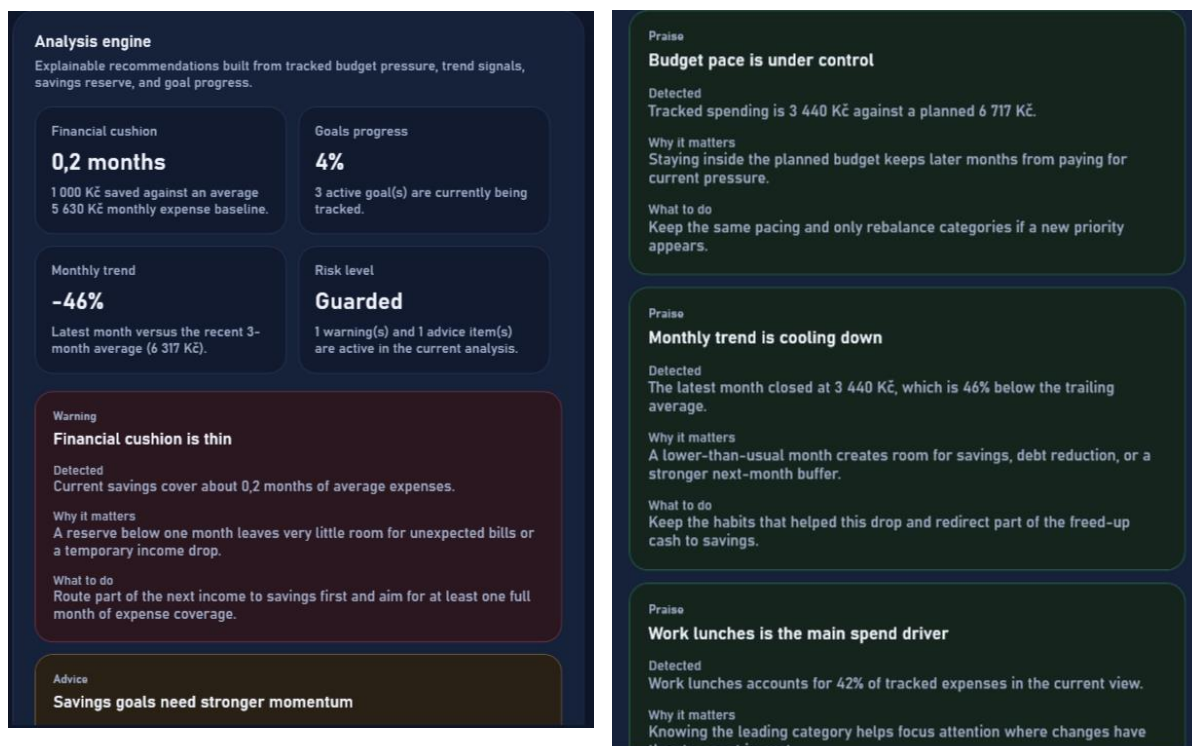
zvoleného kontextu zobrazovat odlišná data i odlišné závěry. Z implementačního hlediska to znamenalo, že dashboard není navázán na jeden pevný zdroj dat, ale průběžně načítá a vyhodnocuje informace podle aktuální relace a vybraného filtru.

Vizuální část dashboardu tvoří několik grafických přehledů. Prvním z nich je Spending mix, který zobrazuje strukturu výdajů podle kategorií. Tento graf pomáhá rychle pochopit, která oblast tvoří největší část rozpočtu a které kategorie mají v daném období největší váhu. Druhým přehledem je Budget pressure, tedy srovnání skutečných výdajů s plánovanými částkami v jednotlivých kategoriích. Výsledkem není jen jednoduché porovnání čísel, ale i rychlé vizuální rozlišení kategorií, které jsou v bezpečném stavu, blíží se limitu nebo jsou již překročené. Třetím prvkem je Monthly trend, který ukazuje vývoj výdajů po měsících a pomáhá zachytit, zda výdaje rostou, klesají nebo se drží na přibližně stejné úrovni.



**Obrázek 23:** Dashboard v personal kontextu se základním finančním přehledem a blokem Budget pulse (zdroj: vlastní)

Vedle grafů obsahuje dashboard také textovou interpretaci finančního stavu. Tato část neslouží jako pouhý doplněk, ale jako snaha převést čísla a grafy do podoby, které uživatel snáze porozumí. Uživatel tak nemusí všechny ukazatele vyhodnocovat pouze sám, ale dostává i stručný komentář k tomu, co je v jeho rozpočtu důležité nebo na co by si měl dát pozor. Tím se dashboard posouvá od pasivního zobrazení dat k aktivní podpoře rozhodování.



**Obrázek 24:** Sekce Analysis engine v personal kontextu (zdroj: vlastní)

Na dashboard navazuje modul explainable analytics engine, který představuje jednu z nejzajímavějších částí celé aplikace. Tento modul není založen na strojovém učení ani na generování náhodných doporučení, ale na sadě předem definovaných pravidel. Systém vyhodnocuje rozpočtové kategorie, trend výdajů, stav úspor, pokrok v savings goals a další ukazatele a na jejich základě vytváří konkrétní závěry. Ty mohou mít podobu Warning, Advice, Observation nebo Praise. Výhodou tohoto přístupu je, že každé doporučení je vysvětlitelné a lze přesně popsat, z jakých dat a pravidel vzniklo. To je důležité nejen z hlediska používání aplikace, ale i z pohledu bakalářské práce, protože tato logika je dobře obhajitelná a formálně popsatelná.

Analytický modul zároveň vytváří několik souhrnných metrik. V dashboardu jsou zobrazovány ukazatele jako Financial cushion, Goals progress, Monthly trend a Risk level. Tyto metriky poskytují uživateli stručný souhrn stavu jeho financí a pomáhají rychleji vyhodnotit, zda je situace stabilní, nebo zda se objevují varovné signály. Zvláště důležitý je ukazatel Risk level, který agreguje výsledky jednotlivých pravidel a převádí je do jednodušší podoby. Uživatel tak nemusí samostatně číst všechny detaily, ale okamžitě vidí, zda je jeho finanční situace vyhodnocena jako nízké, střední nebo vyšší riziko.

Další významnou součástí analytické implementace je Financial Health Score. Tento ukazatel převádí vybrané aspekty finančního chování na hodnotu od 0 do 100. Při výpočtu se zohledňuje například práce s rozpočtem, stabilita výdajů, úspory nebo pokrok v cílech



spoření. Důležité je, že logika výpočtu není pro všechny stejná. U dospělých uživatelů se score opírá především o personal kontext, aby nebyl výsledek zkreslen tím, že uživatel financuje větší část household výdajů. U child účtů je naopak score založeno na jejich shared aktivitě. Tento rozdíl pomáhá zachovat férovější srovnání mezi jednotlivými členy household.

Financial Health Score se nezobrazuje pouze na dashboardu, ale i v části pro správu household members. Tam je doplněn o grafické srovnání členů household, které umožňuje rychle vidět, kdo je v daném období finančně nejstabilnější a kdo naopak zaostává. Tento prvek má nejen analytický, ale i motivační rozměr. V kontextu rodinné aplikace jde o zajímavé rozšíření, které propojuje individuální finanční chování s přehledem celé household skupiny.

Z implementačního hlediska bylo důležité, že dashboard, analytický modul i výpočet score nejsou řešeny izolovaně, ale využívají data z více částí systému současně. Pracují se záznamy, kategoriemi, savings goals, household financemi, personal financemi i aktuální uživatelskou relací. To znamená, že analytika v aplikaci není jen vizuální vrstvou nad jednou tabulkou, ale samostatnou logickou částí, která spojuje více datových zdrojů do jednoho přehledného výstupu.

Celkově lze říci, že dashboard, explainable analytics a Financial Health Score patří mezi nejdůležitější implementační přínosy aplikace My Money. Právě tyto části odlišují výsledné řešení od běžného evidenčního nástroje a posouvají jej směrem k aplikaci, která uživateli nejen ukládá data, ale také mu pomáhá tato data lépe chápat a vyhodnocovat.

## **5.7 Transakce a konzistence dat**

Při implementaci finanční aplikace bylo důležité zajistit nejen správné ukládání dat, ale také jejich vzájemnou konzistenci. V systému My Money totiž řada operací neznamena pouze vytvoření nebo odstranění jednoho záznamu, ale současně ovlivňuje i další části finančního stavu aplikace. Typickým příkladem je přidání finančního záznamu, které může zároveň změnit zůstatek nebo úspory. Pokud by se tyto kroky prováděly odděleně, mohlo by při chybě dojít k tomu, že část dat bude uložena a jiná část nikoli.

Z tohoto důvodu byla do vybraných částí aplikace zavedena podpora databázových transakcí. Transakce zajišťují, že několik navazujících kroků proběhne jako jedna logická operace. Pokud jsou všechny kroky úspěšné, změny se potvrdí. Pokud v některé části nastane chyba, provedené změny se vrátí zpět a databáze zůstane ve stabilním stavu. Tento přístup je u finanční aplikace důležitý, protože i malý nesoulad mezi záznamy a výsledným

finančním stavem by mohl vést k nepřesným přehledům nebo chybným analytickým výstupům.

Transakční zpracování se uplatňuje zejména u operací nad finančními záznamy. Při přidání záznamu je potřeba nejen uložit samotnou položku, ale také upravit odpovídající finance v rámci household nebo personal kontextu. Podobně při odstranění záznamu nestačí smazat jeho záznam z databáze, ale je nutné vrátit související změnu ve zůstatku nebo úsporách. Obě části operace proto musí proběhnout společně.

Konzistence dat je důležitá také v dalších částech aplikace, například při registraci prvního uživatele a vytvoření household nebo u některých operací spojených se správou členů household a jejich finančních struktur. V těchto situacích transakce pomáhají zajistit, aby nevznikly neúplné nebo logicky rozpojené záznamy.

Z hlediska návrhu má transakční zpracování význam i pro servisní vrstvu. Právě ta koordinuje operace složené z více kroků a rozhoduje, které části musí být provedeny společně. Použití transakcí tak přispívá ke stabilitě systému, chrání data před nekonzistencí a zajišťuje, že finanční operace probíhají jako spolehlivé a logicky uzavřené celky.

## 6 TESTOVÁNÍ

Testování představuje důležitou součást vývoje každé softwarové aplikace. U systému My Money mělo testování zvláštní význam nejen proto, že aplikace pracuje s finančními daty, ale také proto, že během vývoje výrazně narostla její složitost. Vedle základní evidence výdajů a úspor bylo totiž nutné ověřit také správné chování víceuživatelského household modelu, role-based omezení, autentizaci, analytické výstupy a konzistenci finančních operací.

Cílem testování bylo ověřit, že aplikace odpovídá stanoveným požadavkům, že se správně chová v běžných scénářích použití a že vybrané části logiky poskytují očekávané výsledky. Zároveň bylo důležité zaměřit se na takové části systému, u nichž by chyba mohla mít přímý dopad na správnost finančních údajů nebo na přístup uživatele k jednotlivým funkcím aplikace.

V rámci projektu bylo testování provedeno ve dvou základních rovinách. První rovinu tvořilo průběžné ověřování funkčnosti jednotlivých částí aplikace při vývoji, zejména pomocí ručního testování uživatelských scénářů. Druhou rovinu tvořily unit testy zaměřené na vybranou část aplikační logiky, konkrétně na analytický modul ExplainableAnalyticsService. Tento přístup byl zvolen proto, že právě analytická pravidla obsahují více podmínek, prahových hodnot a kombinací vstupních dat, u nichž je vhodné chování automatizovaně ověřovat.

V následujících podkapitolách jsou popsány použité metody testování, vybrané testovací scénáře a dosažené výsledky.

### 6.1 Metody testování

Při testování aplikace My Money byly využity dvě základní formy ověření: ruční funkční testování a unit testy. Každá z nich se zaměřovala na jinou část systému.

Ruční funkční testování bylo použito především u částí souvisejících s uživatelským rozhraním a s průchodem aplikací z pohledu běžného uživatele. Tímto způsobem bylo možné ověřit například registraci prvního účtu, přihlášení do systému, přepínání mezi obrazovkami, správu household members, práci s financial records, budget categories, savings goals nebo zobrazení dashboardu. Tento způsob testování byl důležitý i proto, že část chování aplikace závisí na roli uživatele a na aktivním finančním kontextu.

Druhou použitou metodou byly unit testy zaměřené na analytickou logiku služby ExplainableAnalyticsService. Tato část systému pracuje s větším množstvím pravidel a

kombinuje více vstupních údajů, proto bylo vhodné ověřit alespoň vybrané scénáře automatizovaně. Unit testy byly realizovány v samostatném projektu `My_money.Tests`.

Automatizované testování se v současné verzi projektu zaměřuje především na správné vyhodnocení Risk level, interpretaci Monthly trend a tvorbu analytických doporučení. Přestože testovací pokrytí zatím nezahrnuje celý systém, představuje důležitý základ pro další rozšíření testování.

Správnost projektu byla průběžně ověřována také úspěšným sestavením aplikace a kontrolou běhu hlavních částí systému. Tato forma ověření nenahrazuje plnohodnotné automatizované testy, ale potvrzuje, že aplikace tvoří funkční celek.

## 7 ZHODNOCENÍ VÝSLEDKŮ

Výsledná aplikace My Money představuje funkční desktopové řešení zaměřené na správu osobních a rodinných financí. Během práce se projekt postupně rozšířil z jednodušší evidenční aplikace do podoby víceuživatelského household systému, který podporuje role uživatelů, práci s oprávněními, oddělení personal a shared financí, správu cílů spoření, analytický dashboard i základní testování. Výsledné řešení tak odpovídá nejen praktickému použití, ale i rozsahu bakalářské práce zaměřené na návrh a implementaci desktopové aplikace.

Z hlediska funkcionality aplikace pokrývá hlavní oblasti, které byly pro práci stanoveny. Umožňuje registraci a přihlášení uživatelů, správu household členů, evidenci finančních záznamů, práci s rozpočtovými kategoriemi, sledování zůstatku a úspor i správu savings goals. Důležitou součástí řešení je také rozdělení dat podle aktivního kontextu, tedy na household a personal část, a navázání přístupových práv na konkrétní roli uživatele.

Za podstatný přínos výsledné aplikace lze považovat i rozšíření o analytickou vrstvu. My Money neslouží pouze k ukládání finančních údajů, ale nabízí také grafické přehledy, ukazatel Financial Health Score a modul explainable analytics, který převádí finanční data do podoby doporučení a pozorování. Aplikace tak propojuje běžnou evidenci financí s funkcemi, které uživateli pomáhají lépe se orientovat v jeho finanční situaci a sledovat vývoj vlastního hospodaření.

### 7.1 Splnění cílů

Hlavním cílem práce bylo navrhnout a implementovat funkční desktopovou aplikaci pro správu osobních financí a podporu finanční gramotnosti. Tento cíl byl splněn. Výsledná aplikace umožňuje evidenci finančních záznamů, správu budget categories, sledování zůstatku a úspor, práci s savings goals a zobrazení základních i rozšířených finančních přehledů.

Dalším cílem bylo zvolit vhodné technologie a architektonické principy pro realizaci aplikace. Také tento cíl byl splněn. Při vývoji byly použity technologie .NET, C#, WPF a SQLite, architektonický přístup MVVM, Dependency Injection, servisní vrstva a repozitáře. Toto řešení se v praxi ukázalo jako vhodné, protože umožnilo přehledně oddělit jednotlivé části systému a postupně projekt rozšiřovat bez nutnosti zásadní změny základní struktury.

Za důležité lze považovat i to, že aplikace neřeší pouze individuální účetnictví jednoho uživatele, ale podporuje víceuživatelský household model. Tím došlo k rozšíření původního

rozsahu práce o role Admin, Partner a Child, správu členů household, rozdílná oprávnění a oddělení různých finančních kontextů. Výsledné řešení tak ve skutečnosti přesahuje základní podobu jednoduché osobní finanční aplikace.

Za splněný lze považovat také cíl související s analýzou finančního chování. Aplikace obsahuje dashboard, grafické přehledy a rule-based analytický modul, který vyhodnocuje rozpočtové zatížení, vývoj výdajů, pokrok v cílech spoření a další ukazatele. V tomto směru práce naplňuje i požadavek na poskytování doporučení závislých na chování uživatele.

## **7.2 Omezení řešení**

Přestože výsledná aplikace splňuje hlavní cíle práce, má i svá omezení. Především se jedná o lokální desktopovou aplikaci určenou pro operační systém Windows. Řešení tedy není multiplatformní a neumožňuje přístup z mobilních zařízení ani z webového prostředí. Zároveň pracuje s lokální databází SQLite. Taková volba je vhodná pro menší projekt tohoto typu, současně však omezuje možnosti synchronizace dat mezi více zařízeními.

Dalším omezením je skutečnost, že některé bezpečnostní mechanismy jsou řešeny v základní podobě odpovídající rozsahu bakalářské práce. Aplikace například využívá hashování hesel a řízení přístupu podle rolí, ale neobsahuje pokročilejší bezpečnostní prvky, jako je vícefaktorové ověřování, šifrování databázového souboru nebo vzdálená správa účtů.

Určité omezení se týká i testování. V projektu již byly vytvořeny první unit testy pro analytický modul. Jde o důležitý krok správným směrem, testovací pokrytí však zatím nezahrnuje všechny klíčové části systému. Zejména role-based logiku, práci s household daty nebo vybrané finanční operace by bylo vhodné v budoucnu doplnit o širší sadu automatizovaných testů.

Analytický modul je navíc založen na explicitně definovaných pravidlech. To je výhodné z hlediska vysvětlitelnosti a kontroly nad výsledky, ale zároveň to znamená, že systém není adaptivní a neučí se z chování uživatele automaticky. Doporučení jsou tedy srozumitelná a odůvodněná, ale jejich rozsah je omezen pravidly, která byla do aplikace přímo implementována.

## **7.3 Možnosti dalšího rozvoje**

Výsledná podoba aplikace vytváří dobrý základ pro další rozšiřování. Jedním z přirozených směrů dalšího vývoje je rozšíření testování, zejména o další unit testy a případně i integrační testy. To by pomohlo lépe chránit aplikaci před chybami při budoucích úpravách.

Další možností je rozšíření analytického modulu. Do budoucna by bylo možné doplnit další pravidla, nové metriky, pokročilejší porovnávání vývoje výdajů nebo podrobnější doporučení pro jednotlivé členy household. Podobně by bylo možné rozšířit i systém Financial Health Score, například o detailnější rozklad výsledného hodnocení.

Z praktického hlediska by bylo možné aplikaci dále rozvíjet také směrem k vyššímu uživatelskému komfortu. Nabízí se například export dat, rozšíření grafických přehledů, podpora pravidelných plateb, upozornění na překročení rozpočtu nebo připomínky související s cíli spoření. Zajímavým směrem by mohlo být i větší gamifikační zapojení child účtů, které by podpořilo motivační a vzdělávací aspekt aplikace.

Další zajímavou možností rozvoje by bylo propojení aplikace s bankovními službami, aby bylo možné automaticky načítat příjmy a výdaje přímo z bankovního účtu uživatele. Takové řešení by výrazně omezilo nutnost ručního zadávání transakcí a současně by umožnilo průběžnou aktualizaci finančního přehledu na základě reálných bankovních dat. V praxi by však tato funkcionality vyžadovala řešení bezpečné autorizace, ochrany citlivých údajů a komunikace s externími bankovními rozhraními.

Z technického hlediska by další rozvoj mohl zahrnovat přechod na synchronizované nebo cloudové řešení, podporu více zařízení, případně vytvoření webové nebo mobilní verze aplikace. To by však znamenalo výrazně širší rozsah projektu a vyžadovalo by i odlišný přístup k bezpečnosti, databázi a nasazení systému.

Celkově lze shrnout, že aplikace My Money splnila stanovené cíle a zároveň otevřela prostor pro další rozvoj. Výsledné řešení je funkční, architektonicky promyšlené a rozsahem odpovídá bakalářské práci zaměřené na návrh a implementaci moderní desktopové aplikace pro správu financí.

## ZÁVĚR

Cílem bakalářské práce bylo navrhnout a implementovat desktopovou aplikaci zaměřenou na správu osobních financí a podporu finanční gramotnosti uživatele. Tento cíl byl splněn. V rámci práce vznikla aplikace My Money, která umožňuje evidenci finančních záznamů, správu rozpočtových kategorií, sledování zůstatku a úspor, práci s cíli spoření a zobrazení finančních přehledů. Součástí výsledného řešení je také víceuživatelský household model, který rozlišuje role Admin, Partner a Child a umožňuje oddělit osobní a sdílené finance v rámci jedné aplikace.

Při řešení bylo nejprve nutné zpracovat teoretický základ práce, zejména v oblasti platformy .NET, technologie WPF, architektonického vzoru MVVM, datové vazby, Dependency Injection, servisní a repozitářové vrstvy a databáze SQLite. Na tuto část navázala analýza podobných řešení a analýza požadavků, které se staly podkladem pro návrh architektury, datového modelu, uživatelského rozhraní a toku dat v aplikaci. V navazující implementační části byly tyto návrhy převedeny do funkční podoby desktopové aplikace.

Výsledná aplikace nepředstavuje pouze jednoduchý nástroj pro zapisování výdajů, ale řešení, které propojuje evidenci financí s víceuživatelským prostředím a analytickými funkcemi. Důležitou součástí aplikace je dashboard s grafickými přehledy, ukazatel Financial Health Score a modul explainable analytics, který převádí finanční data do podoby srozumitelných doporučení a pozorování. Práce tak ukázala, že i desktopová aplikace s lokální databází může nabídnout přehledné prostředí pro správu financí a současně i rozšířené funkce, které pomáhají uživateli lépe se orientovat v jeho finanční situaci.

Přestože výsledné řešení splňuje hlavní cíle práce, nabízí i prostor pro další rozvoj. Do budoucna by bylo možné rozšířit testování o další automatizované scénáře, doplnit pokročilejší analytická pravidla, rozšířit práci s uživatelskými účty nebo aplikaci propojit s externími bankovními službami. Přínosem této práce je proto nejen samotná realizace funkční aplikace, ale i vytvoření základu, na který je možné v dalších úpravách navázat.



- [1] MICROSOFT. *Overview of .NET Framework*. Microsoft Learn. Online. 2023-03-30. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/framework/get-started/overview>. [cit. 2026-05-09].
- [2] MICROSOFT. *.NET fundamentals documentation*. Microsoft Learn. Online. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/fundamentals/>. [cit. 2026-05-09].
- [3] MICROSOFT. *Introduction to .NET*. Microsoft Learn. Online. 2026-03-13. Dostupné z: <https://learn.microsoft.com/en-gb/dotnet/core/introduction>. [cit. 2026-05-09].
- [4] MICROSOFT. *What is Windows Presentation Foundation*. Microsoft Learn. Online. 2026-03-23. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/>. [cit. 2026-05-09].
- [5] MICROSOFT. *WPF Overview*. Microsoft Learn. Online. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/>. [cit. 2026-05-09].
- [6] GUPTA, Arun. *Building End-to-End Apps with C# 11 and .NET 7: The Complete Guide to Building Web, Desktop, and Mobile Apps*. BPB Publications, 2023. ISBN 978-93-5551-637-4.
- [7] NORTHWOOD, Chris. *The Full Stack Developer*. New York: Apress, 2018. ISBN 978-1-4842-4152-3.
- [8] PETZOLD, Charles. *Applications = Code + Markup: A Guide to the Microsoft Windows Presentation Foundation*. Redmond: Microsoft Press, 2006. ISBN 978-0-7356-1952-3.
- [9] MICROSOFT. *Model-View-ViewModel*. Microsoft Learn. Online. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm/>. [cit. 2026-05-09].
- [10] MICROSOFT. *Data binding overview*. Microsoft Learn. Online. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/data/>. [cit. 2026-05-09].
- [11] MACDONALD, Matthew. *Pro WPF 4.5 in C#: Windows Presentation Foundation in .NET 4.5*. 4th ed. New York: Apress, 2012. ISBN 978-1-4302-4365-6.
- [12] MICROSOFT. *Commanding Overview*. Microsoft Learn. Online. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/advanced/commanding-overview>. [cit. 2026-05-09].

- [13] MARTIN, Robert C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall, 2017. ISBN 978-0-13-449416-6.
- [14] MICROSOFT. *Dependency injection in .NET*. Microsoft Learn. Online. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection>. [cit. 2026-05-09].
- [15] MICROSOFT. *Repository and Unit of Work Patterns*. Microsoft Learn. Online. Dostupné z: <https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions/getting-started-with-ef-5-using-mvc-4/implementing-the-repository-and-unit-of-work-patterns-in-an-asp-net-mvc-application>. [cit. 2026-05-09].
- [16] FOWLER, Martin. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002. ISBN 978-0-321-12742-6.
- [17] SQLITE. *SQLite Documentation*. Online. Dostupné z: <https://www.sqlite.org/docs.html>. [cit. 2026-05-02].
- [18] MICROSOFT. *Microsoft.Data.Sqlite overview*. Microsoft Learn. Online. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/>. [cit. 2026-05-02].
- [19] SQLITE. *Appropriate Uses For SQLite*. Online. Dostupné z: <https://www.sqlite.org/whentouse.html>. [cit. 2026-05-02].
- [20] YNAB. *Features*. Online. 2026. Dostupné z: <https://www.ynab.com/features>. [cit. 2026-05-13].
- [21] BUDGETBAKERS. *Wallet – Personal & Family Finance Manager*. Online. 2026. Dostupné z: <https://budgetbakers.com/en/products/wallet/>. [cit. 2026-05-13].
- [22] SPENDEE. *Shared Wallets*. Online. 2026. Dostupné z: <https://help.spendee.com/article/224-shared-wallets>. [cit. 2026-05-13].
- [23] MONARCH MONEY. *Monarch for Couples*. Online. 2026. Dostupné z: <https://help.monarchmoney.com/hc/en-us/articles/20926382202004-Monarch-for-Couples>. [cit. 2026-05-13].
- [24] GORBACHENKO, Pavel. *What are Functional and Non-Functional Requirements and How to Document These*. Online. Enkonix. 2021. Dostupné z: <https://enkonix.com/blog/functional-requirements-vs-non-functional>. [cit. 2026-05-09].

- [25] OWASP FOUNDATION. *Authentication Cheat Sheet. OWASP Cheat Sheet Series*. Online. Dostupné z: [https://cheatsheetseries.owasp.org/cheatsheets/Authentication\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html). [cit. 2026-05-09].
- [26] MICROSOFT. *Visual Studio documentation. Microsoft Learn*. Online. Dostupné z: <https://learn.microsoft.com/en-us/visualstudio/>. [cit. 2026-05-09].
- [27] MICROSOFT. *NuGet documentation. Microsoft Learn*. Online. Dostupné z: <https://learn.microsoft.com/en-us/nuget/>. [cit. 2026-05-09].
- [28] GIT. *Git Documentation*. Online. Dostupné z: <https://git-scm.com/docs>. [cit. 2026-05-09].
- [29] GITHUB. *GitHub Docs*. Online. Dostupné z: <https://docs.github.com/>. [cit. 2026-05-09].

# PŘÍLOHY

Příloha A – Elektronická příloha

Příloha je uložena v systému STAG a obsahuje:

- Plný zdrojový kód aplikace.
- Tento dokument